Advanced Digital Design

PRESENTED BY:

Dr. Shasanka Sekhar Rout

SYLLABUS

Session 1
Lecture: Basic Digital Circuits
- Basic Digital Circuits: Introduction, Principles, Working.
- Usage / Applications.

Session 2
Lecture: Combinational Circuits
- Logic Arrays
- NAND-NAND / NOR-NOR circuits
- Encoders and Decoders

Session 3
Lecture: Arithmetic Circuits
- 2's Complement
- Half and Full adders
- RCA,CLA,CSA

Session 4 & 5
Lecture: Logic Minimization
- Introduction to Various Logic Minimization techniques
- Quine McCluskey technique
- Shannon's reduction using relay switches

Session 6
Lecture: Logical Hazards
- Hazards and glitches
- Hazard elimination

Session 7
Lecture: Logic Families
- Introduction to different Logic families with their sub-families/types.
- TTL – NAND using TTL with working, different types of TTL families with features, usage.
- ECL – NAND using ECL with working, different types of ECL families with features, usage.
- CMOS – NAND using CMOS with working, Different types of CMOS families with features, usage.
- Comparison of speed and power of different families and sub-families/types.

Session 8
Lecture: Tristate and other circuits
- Tristate
- Muller-C
- AOI (And-Or-Inverter)
- Design of Logic Gates with CMOS technology: NOT, NAND, NOR, XOR.

Session 9
Lecture: Sequential Circuits
- ❖ Sequential Circuit Principles
- ❖ Types of Sequential Circuits
- ❖ Clock jitter, clock skew, timing parameters, propagation delays
- ❖ Latches
- ❖ Flip-flops

Session 10 & 11
Lecture: Counter Design and Frequency Division
- ❖ Counters: Introduction and types
- ❖ Designing of Counters using FSM approach: MOD-N, Binary, Decade
- ❖ Binary Counter as a Frequency divider
- ❖ Frequency division hazards

Session 12
Lecture: Registers
- ❖ Registers
- ❖ Shift Registers
- ❖ LIFO
- ❖ FIFO
- ❖ LFSR

Session 13 & 14
Lecture: Finite State Machines (FSM)
- ❖ Types of FSM
- ❖ Problem solving
- ❖ State reduction techniques

Session 15 & 16
Lecture: Static Timing Analysis
- ❖ Need of Static Timing Analysis of Digital Circuits
- ❖ Metastability
- ❖ Setup and Hold Violations
- ❖ Time Borrowing and Time Stealing

Session 17 & 18
Lecture: Asynchronous Circuits
- ❖ Introduction to multiple clock domain circuits
- ❖ Working across multiple clock domains
- ❖ Handshaking

Session 19 & 20
Lecture: Asynchronous to Synchronous Circuit Interaction

Session 21 & 24
Lecture: Case studies of Advanced Digital Design Circuits
- ❖ Several case studies like complex FSMs, Real life problems and their solutions, Pipelined Approaches for Communication Protocols or Recursive Operations, Design Optimization, Latency reduction, Development of Microporcessor / Microcontroller Design, Circuit Development for Data Encryption / Decryption Algorithms etc.

Session 3

Lecture: Arithmetic Circuits

- ✓2’s Complement
- ✓Half and Full adders
- ✓RCA,CLA,CSA

Arithmetic Circuits

- Arithmetic circuits are fundamental blocks in digital systems and are used for arithmetic operations such as addition, subtraction, multiplication and division.

- There are two complement forms:
  - i. 1's complement form
  - ii. 2's complement form

1's Complement:- The 1's complement of any binary number is performed by simply changing all 1's into 0's and 0's into 1's, i.e. each bit is replaced by its complement.

Ex: 10010 => 01101

2's Complement:- 1's complement of a number + 1.

Ex: 10010 => 01101+1=>01110

Arithmetic Circuits

2's complement Representation:

- If the number is positive, the magnitude is represented in its true binary form and a sign bit 0 is placed in front of the MSB.
- If the number is negative, the magnitude is represented in its 2's complement form and a sign bit 1 is placed in front of the MSB.

Example: Represent the following signed number in 2's complement form.
(a) 25 (b) – 25 (c) 68 (d) – 68

	B_7	B_6	B_5	B_4	B_3	B_2	B_1	B_0
25	0	0	0	1	1	0	0	1
- 25	1	1	1	0	0	1	1	1
68	0	1	0	0	0	1	0	0
- 68	1	0	1	1	1	1	0	0

Arithmetic Circuits

Binary Arithmetic using 2's Complements:

Example 1: Subtract 14 from 46 using 2's complement method.

Solution

+14	=	0 0 0 0 1 1 1 0	
−14	=	1 1 1 1 0 0 1 0	(In 2's complement form)
+46		0 0 1 0 1 1 1 0	
−14	⇒	+1 1 1 1 0 0 1 0	(2's complement form of − 14)
+32		❶ 0 0 1 0 0 0 0 0	(Ignore the carry)

There is a carry, ignore it. The MSB is 0. So, the result is positive and is in normal binary form. Therefore, the result is + 00100000 = + 32.

Binary Arithmetic using 2's Complements:

Example 2: Add -75 to 26 using 2's complement method.

Solution

+75	=	01001011	
−75	=	10110101	(In 2's complement form)
+26		00011010	
−75	⇒	+10110101	(2's complement form of −75)
−49		11001111	(No carry)

There is no carry, the MSB is a 1. So, the result is negative and is in 2's complement form. The magnitude is 2's complement of 11001111, that is, 00110001 = 49. Therefore, the result is −49.

Binary Arithmetic using 2's Complements:

Example 3: Add -14 to -25 using 2's complement method.

Solution

+14 = 00001110

-14 = 11110010 (In 2's complement form)

+25 = 00011001

-25 = 11100111 (In 2's complement form)

-14 11110010 (2's complement form of -14)

-25 => 11100111 (2's complement form of -25)

-39 1 11011001 (Ignore the carry)

There is a carry, ignore it. The MSB is a 1; so the result is negative and is in 2's complement from. The 2's complement of 11011001 is 00100111. Therefore, the result is -39.

ADDERS:

- The most basic arithmetic operation is the addition of two binary digits.
- This simple addition consists of 4 possible operations, namely:

0	0	1	1
+0	+1	+0	+1
			(carry)
0	1	1	1←0

- There are mainly 2 types of adder

a. Half Adder

b. Full Adder

Half Adder:

- A combinational circuit that performs the addition of two bits is called a half adder.
- A half adder is a combinational circuit with two binary inputs (augend and addend bits) and two binary outputs (sum and carry bits).
- It adds the two inputs (A and B) and produces two outputs, sum (S) and carry (C).

Block diagram

Inputs		Outputs	
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

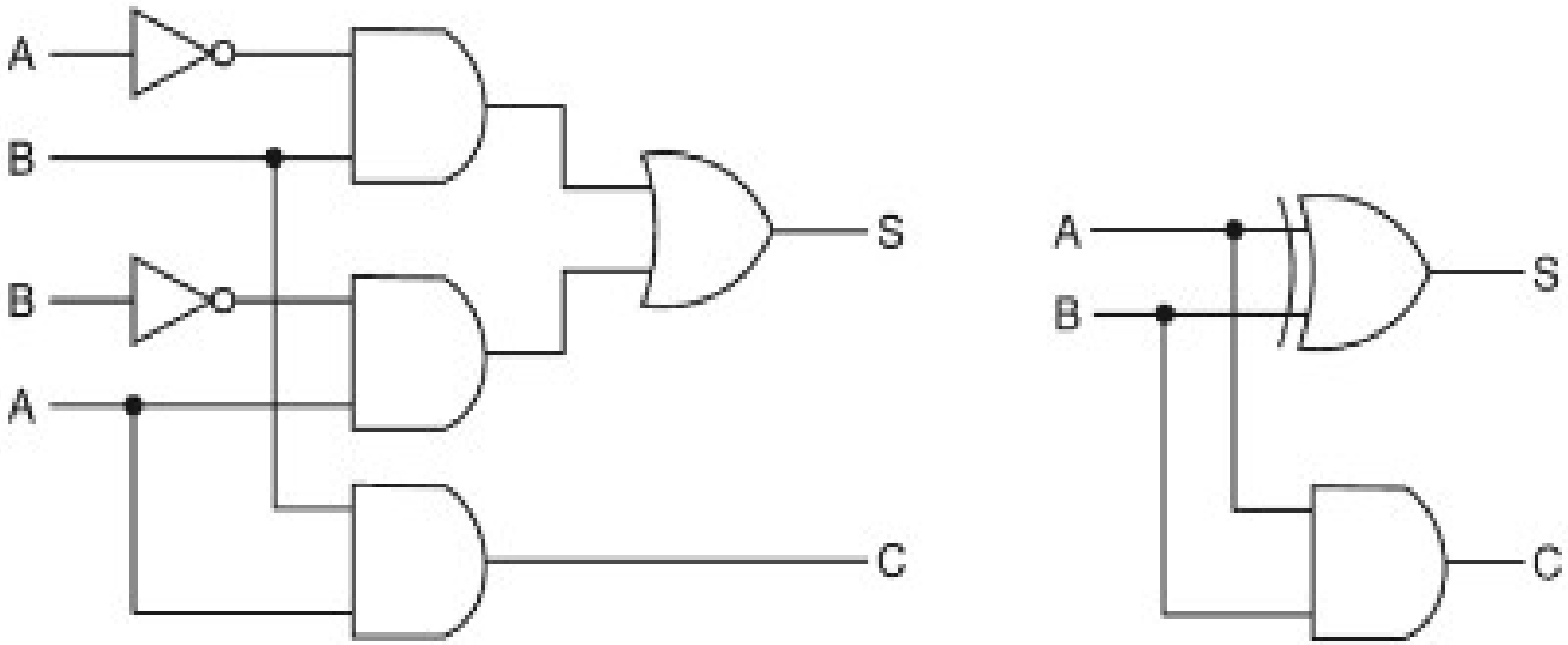
Truth Table

- The simplified Boolean functions for the two outputs can be obtained directly from the truth table. The simplified sum-of-products expressions are

$$S = \overline{A}B + A\overline{B} = A \oplus B$$

$$C = AB$$

Half Adder:



Logic diagrams of half-adder.

Full Adder:

- A full adder is a combinational circuit that forms the arithmetic sum of three bits (two significant bits and a previous carry).
- It consists of three inputs and two outputs.
- The full adder adds the bits A and B and the carry from the previous column called the carry-in C_{in} and outputs the sum bit S and the carry bit called the carry-out C_{out}.

Block diagram

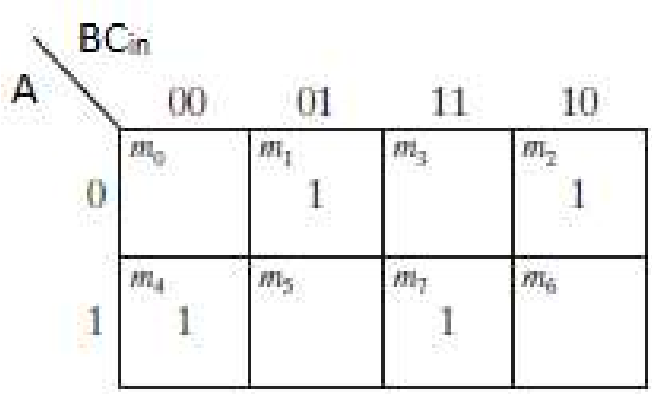
Inputs			Outputs	
A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Truth Table

Full Adder:

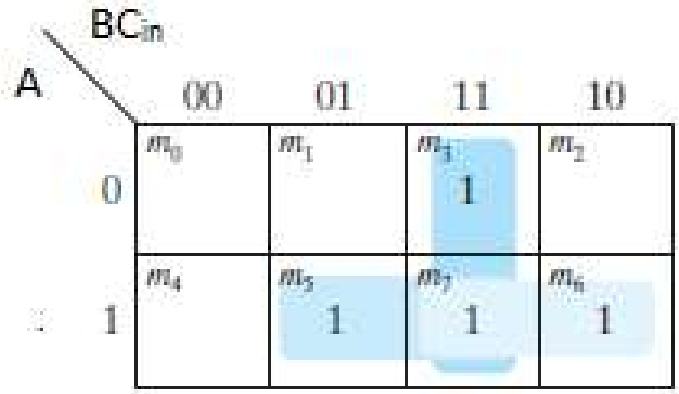
- The simplified expressions are

$S = \sum(1, 2, 4, 7)$



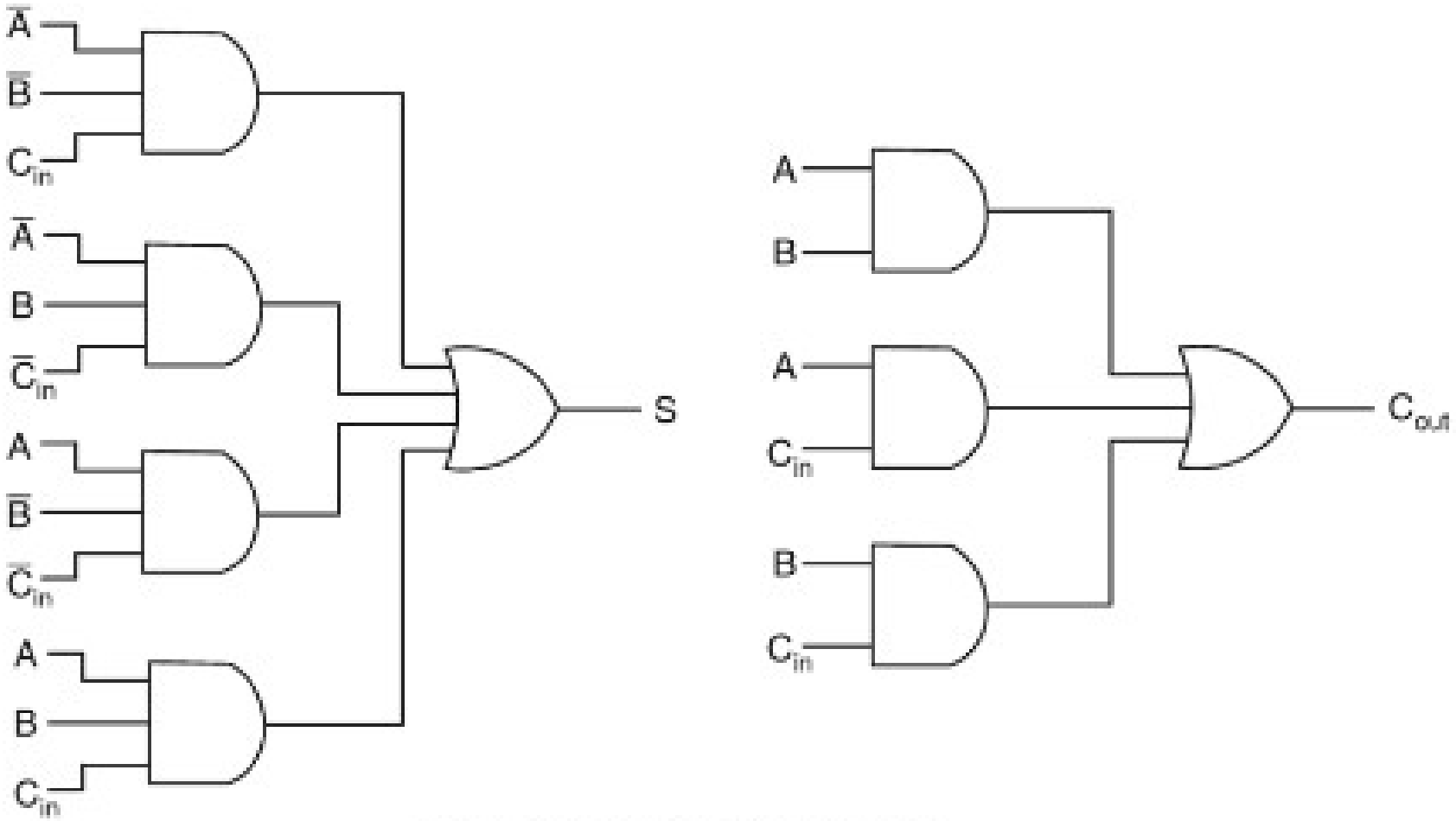
$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$

$C_{out} = \sum(3, 5, 6, 7)$



$$C_{out} = AB + AC_{in} + BC_{in}$$

Full Adder:



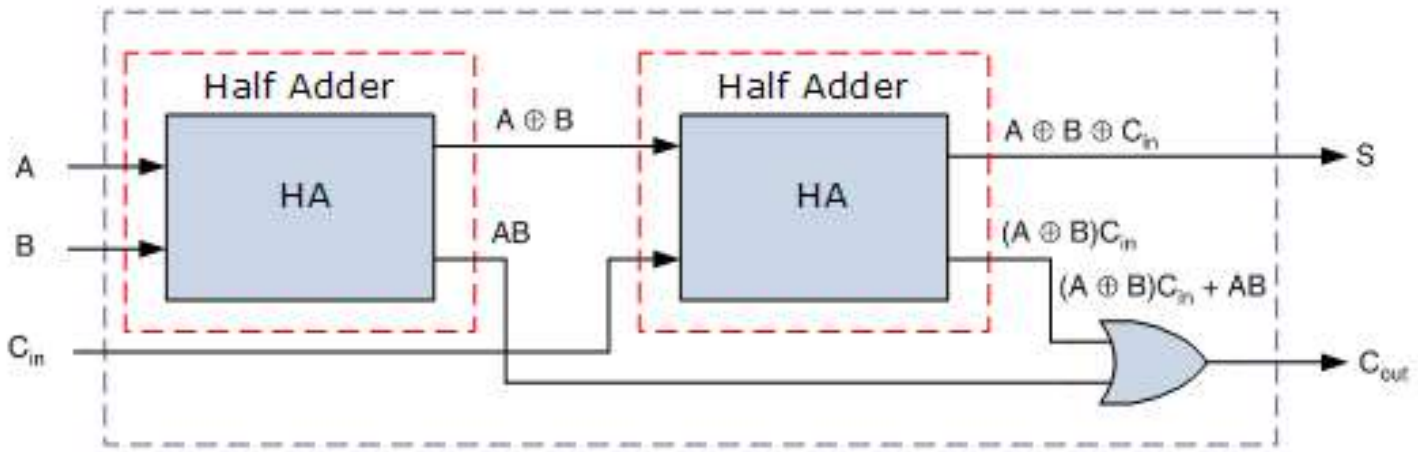
Logic diagrams of full-adder.

Implementation of full adder with two half adders and an OR gate:

From the truth table of full adder, the outputs sum and carry are described by:

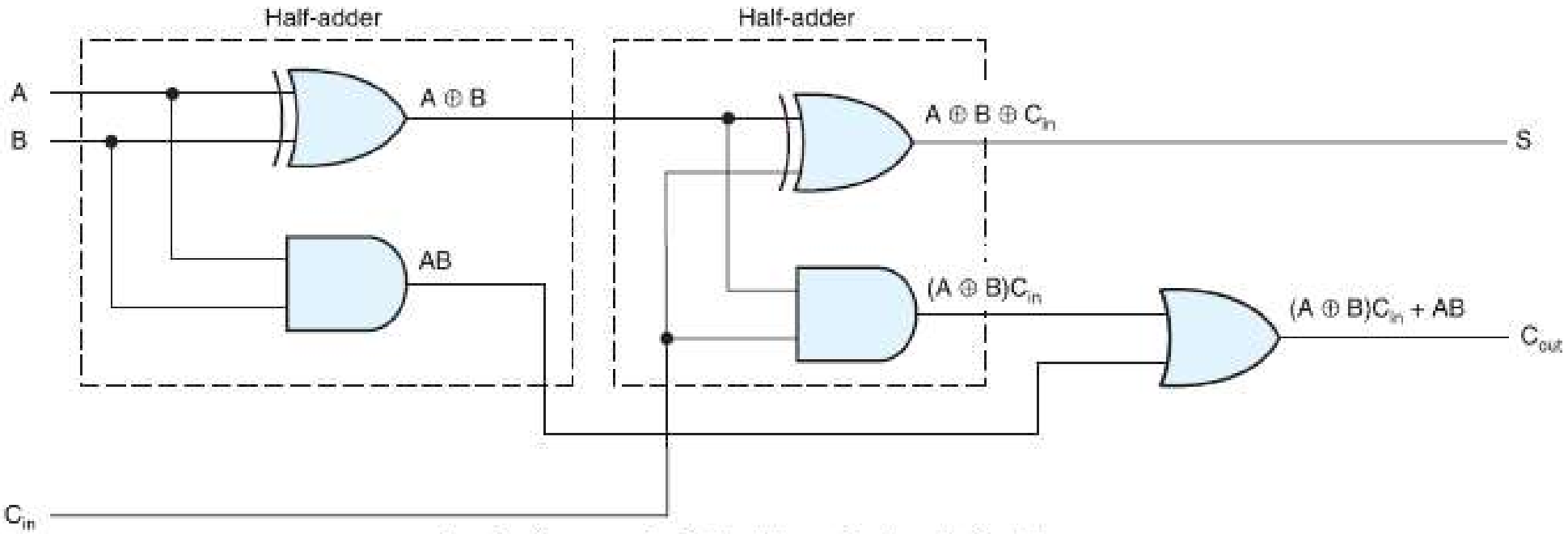
$$S = \bar{A}\bar{B}C_{in} + \bar{A}B\bar{C}_{in} + A\bar{B}\bar{C}_{in} + ABC_{in}$$
$$= (A\bar{B} + \bar{A}B)\bar{C}_{in} + (AB + \bar{A}\bar{B})C_{in}$$
$$= (A \oplus B)\bar{C}_{in} + \overline{(A \oplus B)}C_{in} = A \oplus B \oplus C_{in}$$

$$C_{out} = \bar{A}BC_{in} + A\bar{B}C_{in} + AB\bar{C}_{in} + ABC_{in}$$
$$= (\bar{A}B + A\bar{B})C_{in} + AB(\bar{C}_{in} + C_{in}) = AB + (A \oplus B)C_{in}$$



Block diagram of a full-adder using two half-adders.

Implementation of full adder with two half adders and an OR gate:



Logic diagram of a full-adder using two half-adders.

4-bit Binary Adder/Ripple Carry Adder (RCA):

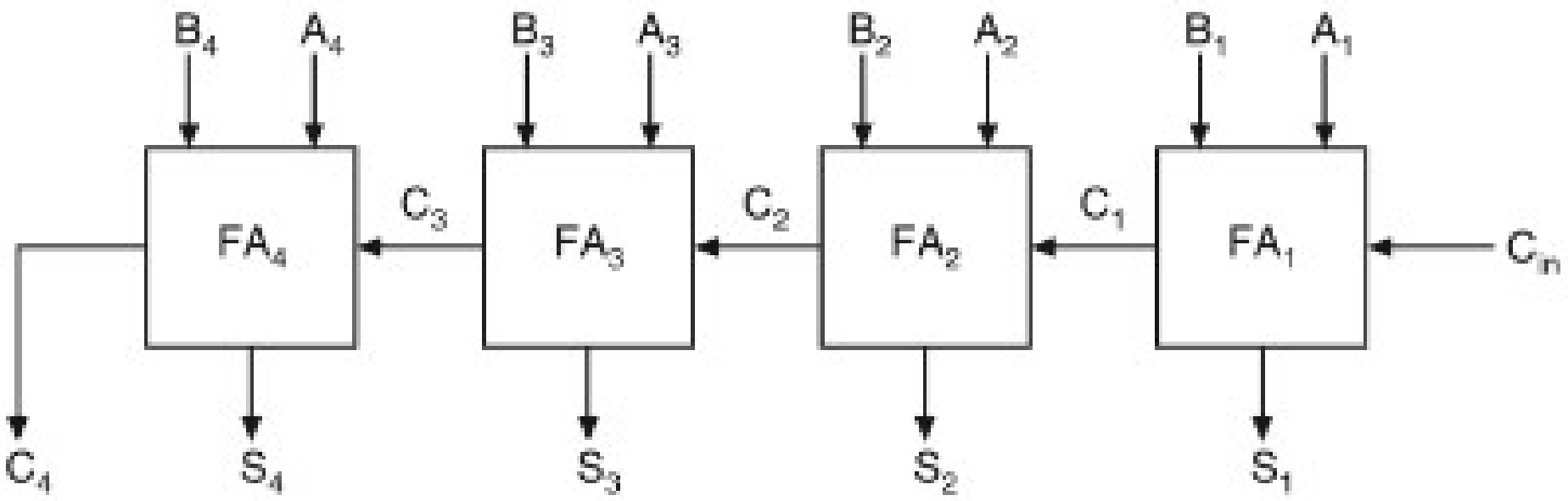
- A binary adder is a digital circuit that produces the arithmetic sum of two binary numbers.
- It can be constructed with full adders connected in cascade, with the output carry from each full adder connected to the input carry of the next full adder in the chain.

Example: Consider the two binary numbers A = 1011 and B = 0011. Their sum S = 1110 is formed with the four-bit adder as follows:

Subscript i:	3	2	1	0	
Input carry	0	1	1	0	C_i
Augend	1	0	1	1	A_i
Addend	0	0	1	1	B_i
Sum	1	1	1	0	S_i
Output carry	0	0	1	1	C_{i+1}

4-bit binary adder/RCA:

- Figure shows the interconnection of four full-adder (FA) circuits to provide a 4-bit binary parallel adder.



Logic diagram of a 4-bit binary parallel adder.

- The parallel adder, in which the carry-out of each full-adder is the carry-in to the next most significant adder is called a ripple carry adder.

Carry Look Ahead Adder (CLA):

- Binary adder produces a longest propagation delay due to carry bit.
- The signal from C_i to the output carry C_{i+1} propagates through an AND and OR gates. Therefore, for an n-bit Ripple Carry Adder, there is a delay carry to propagate from input to output.
- A carry look-ahead adder (CLA) is an electronic adder used for binary addition. Due to the quick additions performed, it is also known as a fast adder.
- The CLA logic uses the concepts of generating and propagating carries. We can say that the CLA adder is the successor of the Ripple Carry Adder.

A	B	C_{in}	Sum	Carry	Condition
0	0	0	0	0	No Carry Generated
0	0	1	1	0	
0	1	0	1	0	
0	1	1	0	1	Carry Propagated
1	0	0	1	0	No Carry
1	0	1	0	1	Carry Propagated
1	1	0	0	1	Carry Generated
1	1	1	1	1	

Carry Look Ahead Adder (CLA):

- Let us now consider two new variables, **Carry Generate (Gi)** and **Carry Propagate (Pi)**.
- For case 1, we see that an output carry is propagated, when we give an input carry. We will refer to this with Pi. So, the mathematical expression of Pi can we represented as :

$$\mathbf{Pi = Ai \oplus Bi}$$

- While considering case 2, we see that an output carry is generated when both inputs, A and B, are high, regardless of the value of the input carry. We will refer to this output carry as Gi. Thus, we can mathematically express Gi as :

$$\mathbf{G_i = A_i \,.\, B_i}$$

- Originally, for a full adder we have the following equations:

$$\mathbf{Sum = A \oplus B \oplus Ci}$$

$$\mathbf{Carry = Ci(A \oplus B) + AB}$$

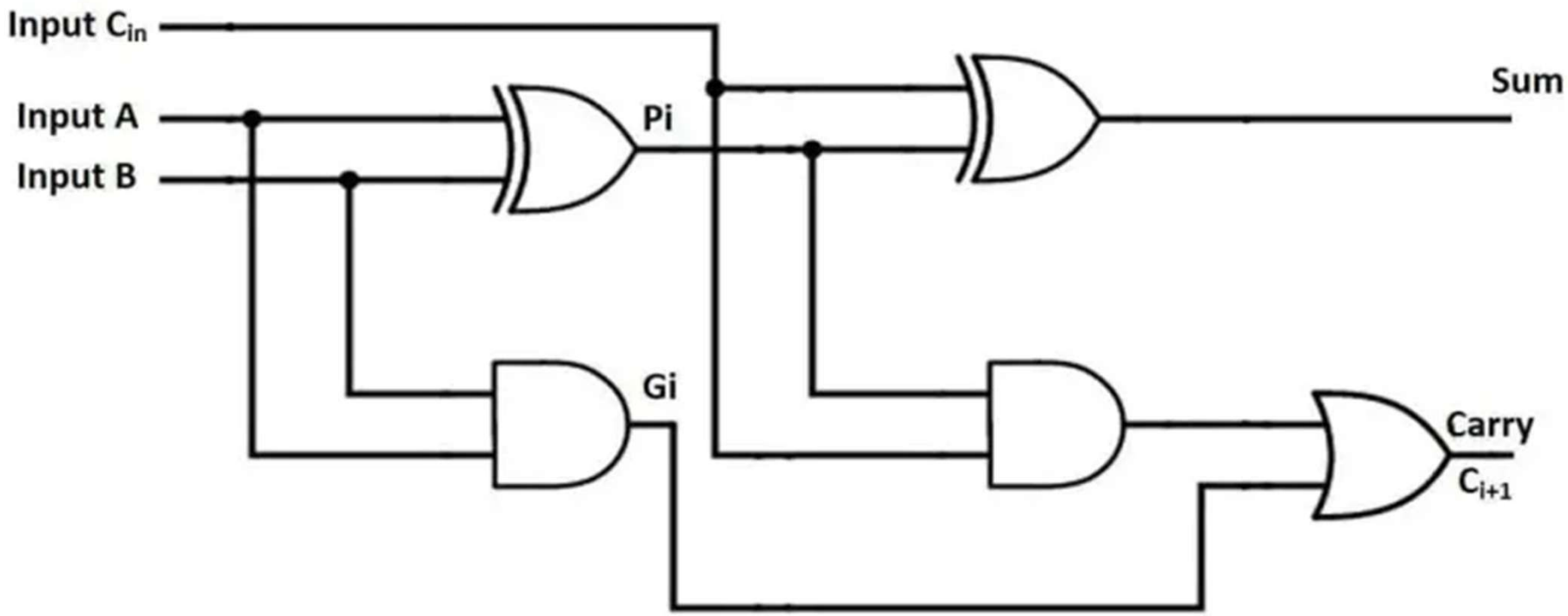
- Thus, we can rewrite the equations of the full adder in terms of Carry Propagate (Pi) and Carry Generate (Gi) as :

$$\mathbf{Sum = Pi \oplus Ci}$$

$$\mathbf{Carry = Gi + Pi \,.\, Ci}$$

Carry Look Ahead Adder (CLA):

- The equations of Sum and Carry can be represented by a logic circuit given below.



4-bit CLA Adder:

We can calculate the output carry C1, C2, C3, and C4 using the above derived equations as:

$C1 = \underline{\mathbf{(C_{in} . P0) + G0}}$

$C2 = (C1 . P1) + G1 = (((C_{in} . P0) + G0) . P1) + G1$

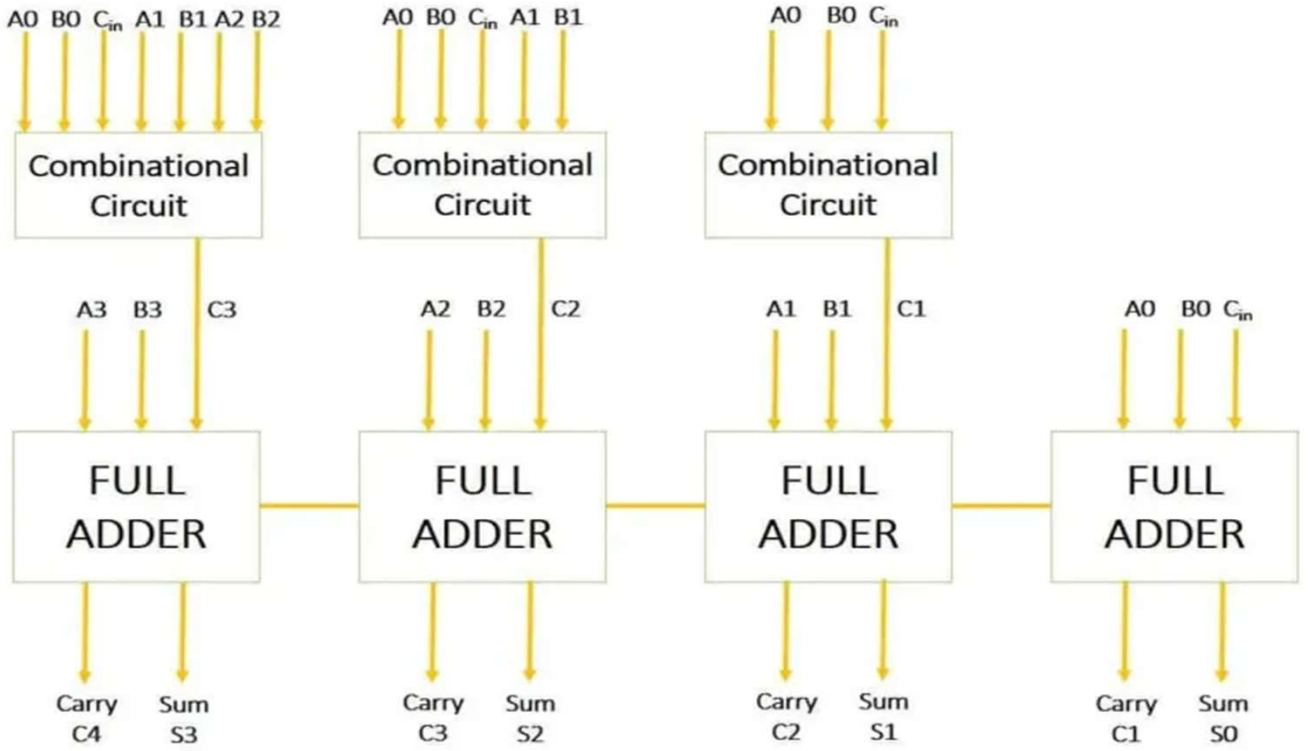
$= \underline{\mathbf{(C_{in} . P0 . P1) + (G0 . P1) + G1}}$

$C3 = (C2 . P2) + G2 = (((C1 . P1) + G1) . P2) + G2$

$= \underline{\mathbf{G2 + (P2 . G1) + (P2 . P1 . G0) + (P2 . P1 . P0 . C_{in})}}$

$C4 = (C3 . P3) + G3$

$= \underline{\mathbf{(C_{in} . P0 . P1 . P2 . P3) + (P3 . P2 . P1 . G0) + (P3 . P2 . G1) + (G2 . P3) + G3}}$

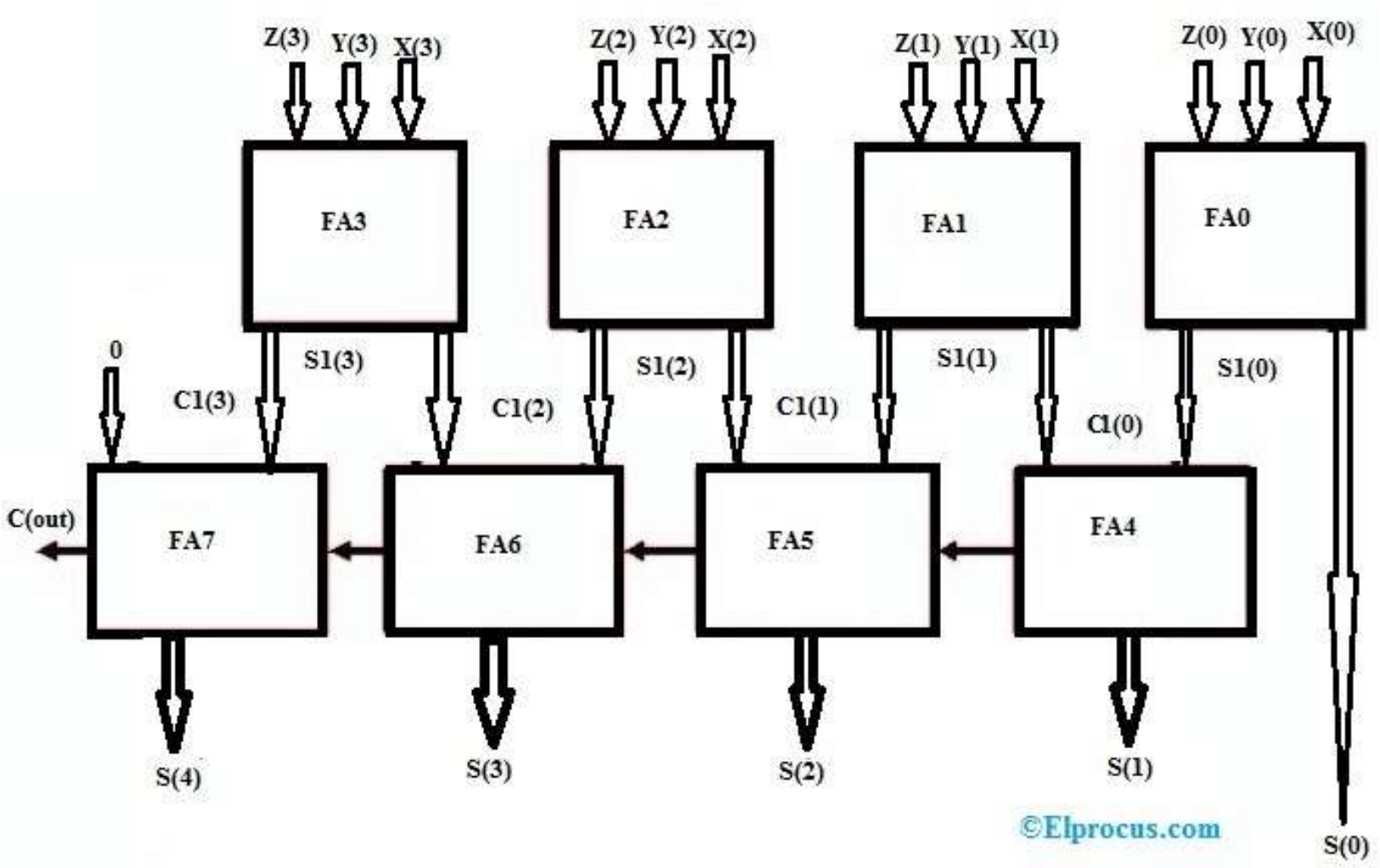


Carry Save Adder (CSA):

- ✓ A Carry Save Adder (CSA) is a type of adder circuit used to efficiently add multiple binary numbers.
- ✓ Unlike traditional adders like Ripple Carry Adder (RCA) or Carry Lookahead Adder (CLA), a CSA reduces the propagation delay by avoiding immediate carry propagation, making it ideal for high-speed arithmetic operations in VLSI design.
- A CSA is based on the idea of computing the sum and carry separately without immediately propagating the carry.
- It uses a **full adder (FA) array** to process three input numbers and generates two outputs:
- **Sum bits (S)** (without considering carries)
- **Carry bits (C)** (shifted left for the next stage)
- These two outputs are then passed to another adder stage, such as a Ripple Carry Adder (RCA) or Carry Lookahead Adder (CLA), to obtain the final sum.

Carry Save Adder (CSA):

- ✓ CSA is very different as compared to other types because it doesn't transmit the middle carries toward the next stages, but in its place, it saves the carry & addends to the sum of the next stage with another fuller adder (FA).



Carry Save Adder (CSA):

Advantages of CSA in VLSI

- **Reduces Carry Propagation Delay:** Since carry bits are saved instead of being propagated immediately, speed improves.
- **Efficient for Multi-Operand Addition:** Used in multipliers and MAC (Multiply-Accumulate) units.
- **Scalable for Large Bit Widths:** Can be combined in Wallace Tree or Dadda Tree multipliers for parallel processing.
- **Used in High-Performance ALUs:** Optimizes operations in processors and digital signal processing (DSP).

Disadvantages of CSA

- It is very efficient only for high-bit operations.
- It has high power consumption & propagation delay for few-bit operations.
- This type of adder does not solve the problem of adding 2 integers to generate a single output. In its place, it simply adds 3 integers & generates two, so the sum of two integers is equivalent to the three inputs sum.

Session 4 & 5

Lecture: Logic Minimization

- ✓ Introduction to Various Logic Minimization techniques
- ✓ Quine McCluskey technique
- ✓ Shannon's reduction using relay switches

Logic Minimization

- Logic minimization is the process of reducing the complexity of a Boolean function while maintaining its functionality.
- This is crucial in **VLSI design** to optimize circuit performance, reduce power consumption, and minimize the number of logic gates.

Introduction to Various Logic Minimization techniques:

- ✓ Algebraic Simplification
- ✓ Karnaugh Map (K-Map)
- ✓ Quine-McCluskey Method

Algebraic Simplification:

- Algebraic simplification reduces Boolean expressions using **Boolean algebra laws** to achieve a minimal representation.
- This helps optimize digital circuits by reducing the number of gates and transistors required.

Ex: (x + y)(x + y')

$$= xx + xy' + xy + yy' = x + xy + xy' + 0 = x(1 + y + y') = x$$

xy + x'z + yz

$$= xy + x'z + yz(x + x') = xy + x'z + xyz + x'yz = xy(1 + z) + x'z(1 + y) = xy + x'z$$

Logic Minimization

Karnaugh Map (K-Map):

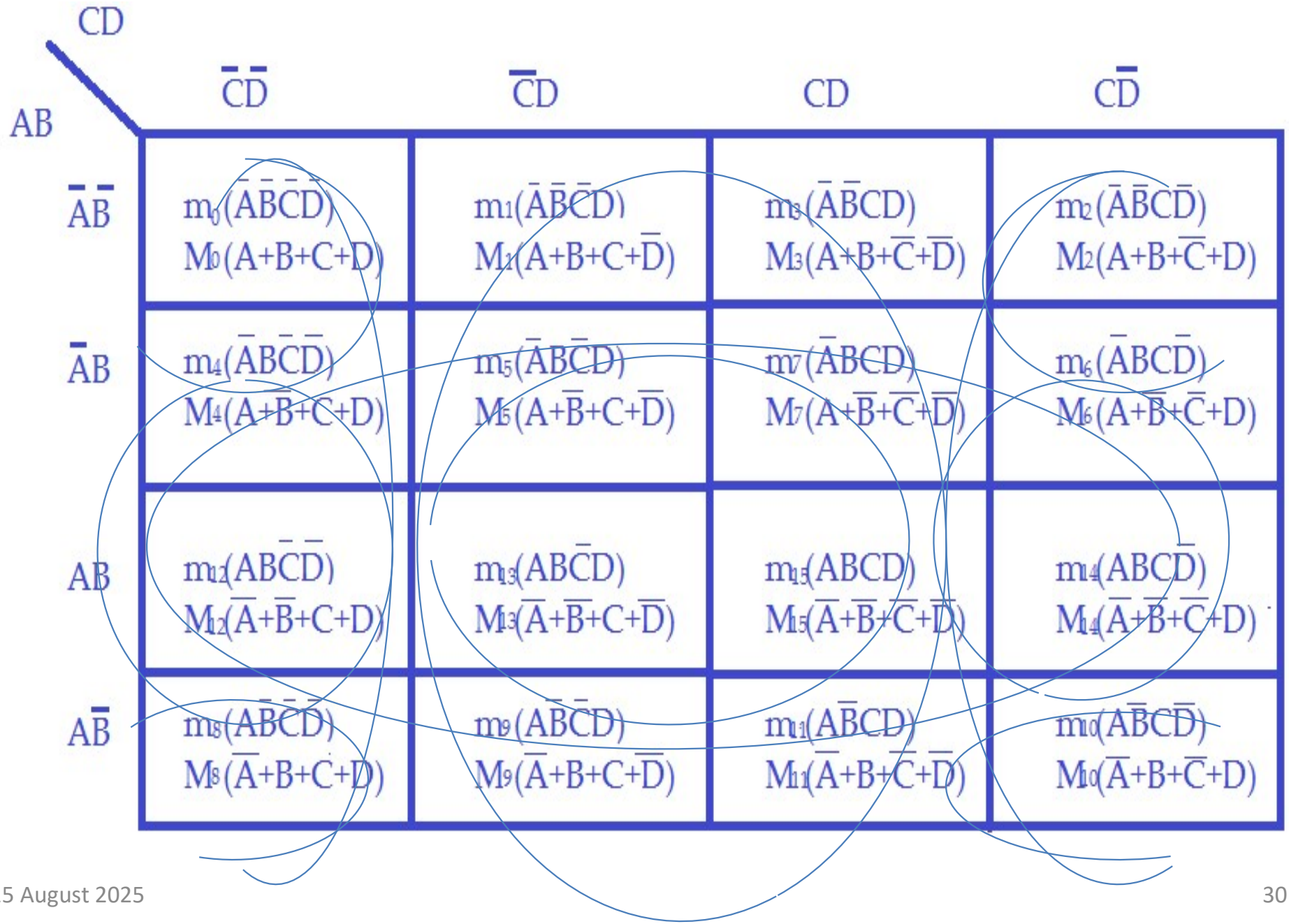
- Proposed by Maurice Karnaugh in 1953. (Also known as Veitch representation)
- A graphical technique for finding the SOP or POS simplification of a boolean expression.
- Modified/pictorial representation of the truth table.
- A square or rectangle divided into smaller squares called cells, each of which represents one minterm or maxterm of the boolean expression to be mapped.
- For n variable(s) : 2^n square cells.
- ✓ Groupings can contain only 1s for SOP and only 0s for POS simplification.
- ✓ Groups can be formed only at right angles, diagonal groups are not allowed.
- ✓ The number of 1s in a group must be a power of 2.
- ✓ Groups can overlap and wrap around the sides of the K map.
- ✓ Follows 2 identities : x + x' = 1 or x . x' = 0

Types of Simplification

- 2 Variables K Map
- 3 Variables K Map
- 4 Variables K Map

AB \ CD	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	$m_0(\bar{A}\bar{B}\bar{C}\bar{D})$ $M_0(A+B+C+D)$	$m_1(\bar{A}\bar{B}\bar{C}D)$ $M_1(A+B+C+\bar{D})$	$m_3(\bar{A}\bar{B}CD)$ $M_3(A+B+\bar{C}+\bar{D})$	$m_2(\bar{A}\bar{B}C\bar{D})$ $M_2(A+B+\bar{C}+D)$
$\bar{A}B$	$m_4(\bar{A}B\bar{C}\bar{D})$ $M_4(A+\bar{B}+C+D)$	$m_5(\bar{A}B\bar{C}D)$ $M_5(A+\bar{B}+C+\bar{D})$	$m_7(\bar{A}BCD)$ $M_7(A+\bar{B}+\bar{C}+\bar{D})$	$m_6(\bar{A}BC\bar{D})$ $M_6(A+\bar{B}+\bar{C}+D)$
AB	$m_{12}(AB\bar{C}\bar{D})$ $M_{12}(\bar{A}+\bar{B}+C+D)$	$m_{13}(AB\bar{C}D)$ $M_{13}(\bar{A}+\bar{B}+C+\bar{D})$	$m_{15}(ABCD)$ $M_{15}(\bar{A}+\bar{B}+\bar{C}+\bar{D})$	$m_{14}(ABC\bar{D})$ $M_{14}(\bar{A}+\bar{B}+\bar{C}+D)$
$A\bar{B}$	$m_8(A\bar{B}\bar{C}\bar{D})$ $M_8(\bar{A}+B+C+D)$	$m_9(A\bar{B}\bar{C}D)$ $M_9(\bar{A}+B+C+\bar{D})$	$m_{11}(A\bar{B}CD)$ $M_{11}(\bar{A}+B+\bar{C}+\bar{D})$	$m_{10}(A\bar{B}C\bar{D})$ $M_{10}(\bar{A}+B+\bar{C}+D)$

Grouping Arrangement



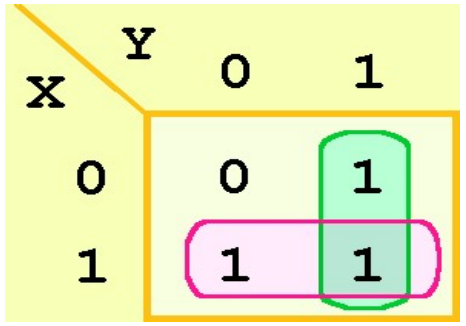
Examples For SOP

- **2 variables K map**

$F(x,y) = x'y + xy' + xy$

$= \Sigma m(1,2,3)$

$= x + y$

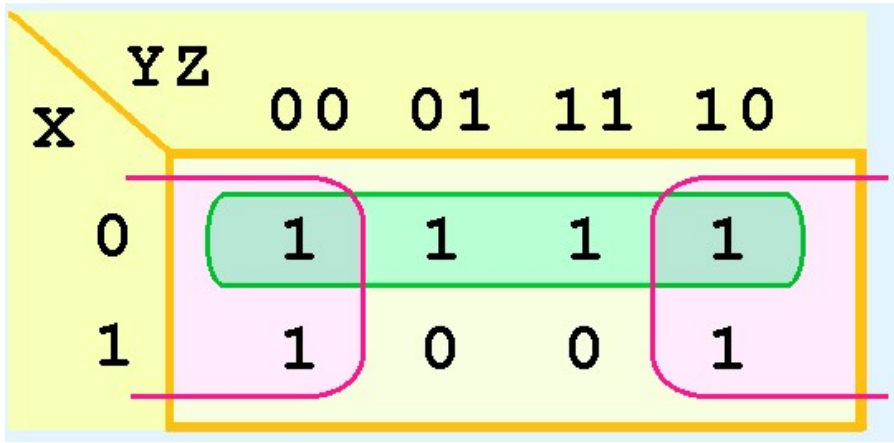


- **3 variables K map**

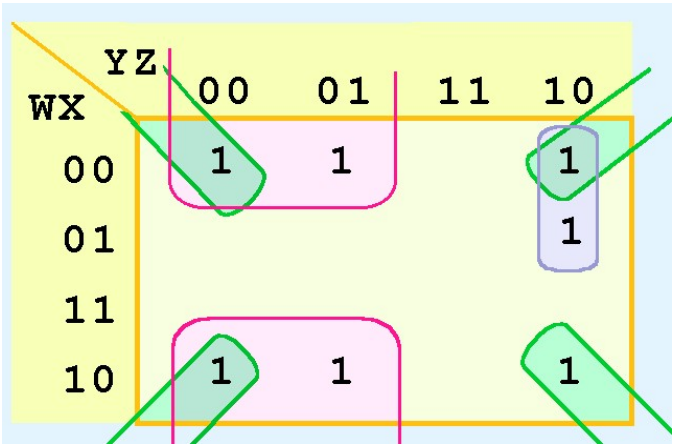
$F(x,y,z) = x'y'z' + x'y'z + x'yz + x'yz' + xy'z' + xyz'$

$= \Sigma m(0,1,2,3,4,6)$

$= x' + z'$



❖ 4 variables K map



$F(w,x,y,z) = \Sigma m(0,1,2,6,8,9,10)$

The simplified function will be

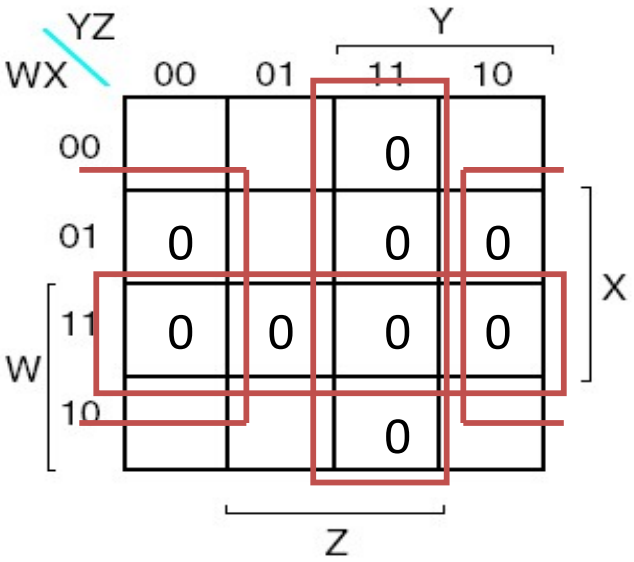
$F(w,x,y,z) = x'y' + x'z' + w'yz'$

❖ Example of 4 variables K map for POS Expression

$F(w,x,y,z) = \prod M(3,4,6,7,11,12,13,14,15)$

The simplified function is

$F(w,x,y,z) = (w'+x')(y'+z')(x'+ z)$

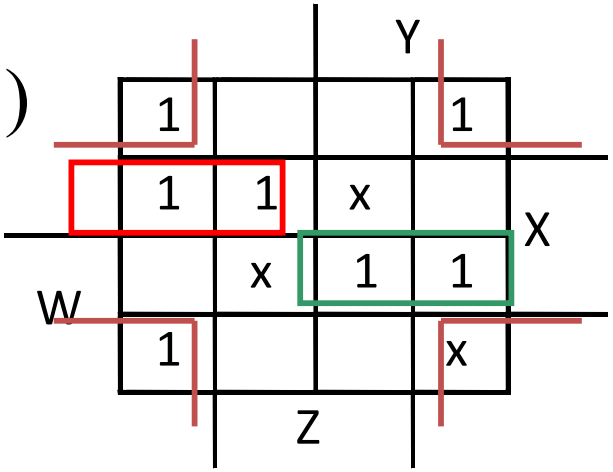


Don’t Care Conditions

- In a K map, a don’t care condition is identified by an X (0 or 1) in the cell of the minterm(s) or maxterm(s) for the don’t care inputs for the most simplification.

$F(w,x,y,z) = \Sigma m(0,2,4,5,8,14,15) +$

$d(w,x,y,z) = \Sigma m(7,10,13)$



The simplified function is

$f(w,x,y,z) = x'z' + w'xy' + wxy$

Logic Minimization

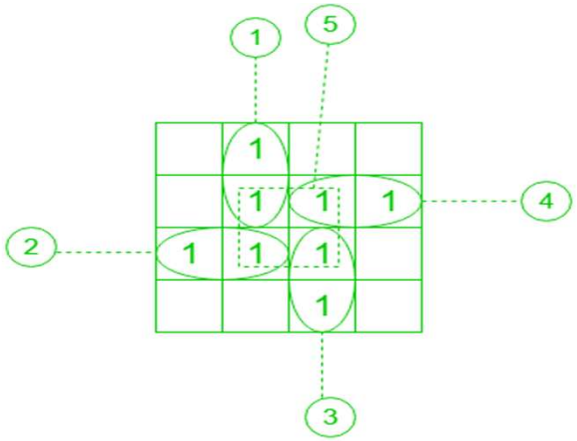
Karnaugh Map (K-Map):

- **Implicant** is a product/minterm term in Sum of Products (SOP) or sum/maxterm term in Product of Sums (POS) of a Boolean function.
- A P**rime Implicant (PI)** is a product term obtained by combining the maximum possible number of adjacent squares in the map.
- **Explicit Implicants (EI)** represent the minimal set of terms needed to cover all the output 1s in the truth table.
- **Essential Prime Implicants (EPI)** are those subcubes (groups) that cover at least one minterm that can't be covered by any other prime implicant.
- These are those prime implicants that always appear in the final solution.
- **Redundant Prime Implicants (RPI)** are the prime implicants for which each of its minterm is covered by some essential prime implicant.
- This prime implicant never appears in the final solution.
- **Selective Prime Implicants (SPI)** are the prime implicants for which are neither essential nor redundant prime implicants.
- These are also known as non-essential prime implicants.

Logic Minimization

Karnaugh Map (K-Map):

Example 1: Given F = ∑(1, 5, 6, 7, 11, 12, 13, 15), find number of implicant, PI, EPI, RPI, SPI.



No. of Implicants = 8

PI = (1,2,3,4,5)

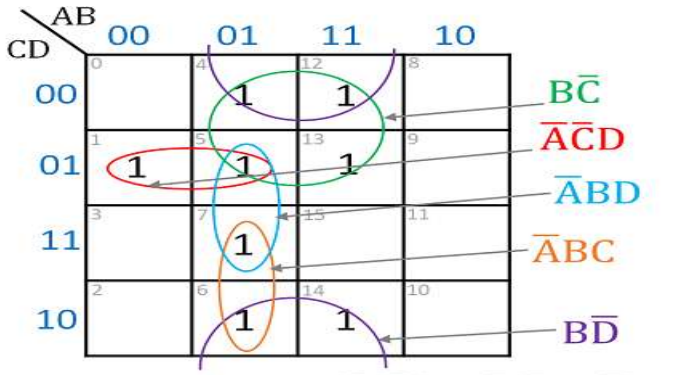
EPI = (1,2,3,4)

RPI = (5)

```
Expression : BD + A'C'D + A'BC+ ACD+ABC'

No. of Implicants = 8
No. of Prime Implicants(PI) = 5
No. of Essential Prime Implicants(EPI) = 4
No. of Redundant Prime Implicants(RPI) = 1
No. of Selective Prime Implicants(SPI) = 0
```

F = (1) + (2) + (3) + (4)



Prime implicants: $B\bar{D}$, $\bar{A}\bar{C}D$, $B\bar{C}$, $\bar{A}BC$, $\bar{A}BD$

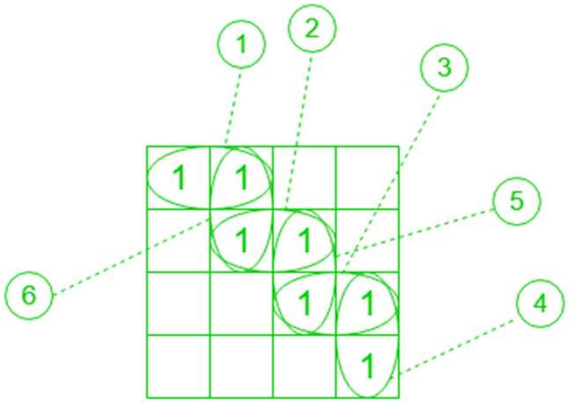
Essential prime implicants: $B\bar{D}$, $\bar{A}\bar{C}D$, $B\bar{C}$

$F = B\bar{D} + \bar{A}\bar{C}D + B\bar{C} + \bar{A}BC$

Logic Minimization

Karnaugh Map (K-Map):

Example 2: Given F = ∑(0, 1, 5, 7, 15, 14, 10), find number of implicant, PI, EPI, RPI and SPI.



No. of Implicants = 7

PI = (1,2,3,4,5,6)

EPI = (1,4)

SPI = (2,3,5,6)

F = (1) + (2) + (3) + (4)

OR

F = (1) + (5) + (6) + (4)

```
No. of Implicants = 7

No. of Prime Implicants(PI) = 6

No. of Essential Prime Implicants(EPI) = 2

No. of Redundant Prime Implicants(RPI) = 0

No. of Selective Prime Implicants(SPI) = 4
```

Logic Minimization

Quine-McCluskey Method/Technique:

- The **Quine-McCluskey method** is a systematic way to minimize Boolean expressions, particularly useful for functions with **more than 4 variables**, where Karnaugh maps become difficult to use.
- It is an algorithmic approach that finds all **prime implicants** and selects the essential ones to create a minimized expression.

Steps in the Quine-McCluskey Method:

1. Convert the Boolean function to minterms (Σ notation)

- Identify the minterms where the function is **1**.
- Express it in decimal form.

2. Convert minterms to binary form

- Represent each minterm using an **n-bit binary number** (where **n** is the number of variables).

3. Group minterms based on the number of 1's

- Arrange minterms into groups based on how many **1s** are present in their binary representation.

Logic Minimization

4. Compare and Combine Adjacent Terms

- If two terms differ by **only one bit**, replace the differing bit with a **dash (-)**.
- Continue this process iteratively until no further simplifications are possible.
- The remaining terms are **Prime Implicants**.

5. Construct a Prime Implicant Chart

- List all **prime implicants** and check which minterms they cover.
- Identify **Essential Prime Implicants** (those that cover minterms not covered by others).

6. Select the Minimal Cover

- Use the **smallest number of prime implicants** that still cover all minterms.

Advantages:

✓ Systematic and algorithmic → Works well for large functions.
✓ This method is suitable for a large number of inputs(n>4) for which K-map construction is a tedious task.
✓ Can be implemented in software tools for logic synthesis.

Disadvantages:

✕ Computationally expensive for very large numbers of variables.
✕ Exponential complexity as the number of minterms increases.

Logic Minimization

Example: Minimize the given function by Quine-McCluskey method
$F(A,B,C,D) = \sum m(0,1,2,4,6,8,9,11,13,15)$

Solution:

Step 1: Convert the given minterms into their binary representation and arrange them according to the number of ones present in the binary representation.

TABLE 1

Group	Minterm	A	B	C	D
0	0	0	0	0	0
1	1	0	0	0	1
	2	0	0	1	0
	4	0	1	0	0
	8	1	0	0	0
2	6	0	1	1	0
	9	1	0	0	1
3	11	1	0	1	1
	13	1	1	0	1
4	15	1	1	1	1

Logic Minimization

Step 2: Take minterms from successive groups(simultaneous group only) which have an only a 1-bit difference in their representation and form their pair by merging them and making a group of the pairs which are from the same groups that are merged (for example 0 is from group 0 and 1 is from group 1 so it is added to the group 0.

TABLE-2

Group	Pair	A	B	C	D
0	(0,1)	0	0	0	–
	(0,2)	0	0	–	0
	(0,4)	0	–	0	0
	(0,8)	–	0	0	0
1	(1,9)	–	0	0	1
	(2,6)	0	–	1	0
	(4,6)	0	1	–	0
	(8,9)	1	0	0	–
2	(9,11)	1	0	–	1
	(9,13)	1	–	0	1
3	(11,15)	1	–	1	1
	(13,15)	1	1	–	1

Logic Minimization

Step 3: repeat the same step by taking pairs of successive groups merging them where there is only a 1-bit difference and keeping them in groups according to the groups from where they are merged and placed – in bit difference.

TABLE-3

Group	Quad	A	B	C	D
0	(0,1,8,9)	–	0	0	–
	(0,2,4,6)	0	–	–	0
1	(9,11,13,15)	1	–	–	1

- ✓ After table 3 the process is stopped as there is no 1-bit difference in the remaining group minterms in the simultaneous groups of table 3.

- ✓ Now, the remaining quads present in table 3 represent the prime implicants for the given boolean function. So, we construct prime implicants table which contains the obtained prime implicants as rows and the given minterms as columns.

Logic Minimization

Step 4: Place 1 in the corresponding place which the minterm can represent. Add the minterm to the simplified boolean expression if the given minterm is only covered by this prime implicant.

PRIME IMPLICANT TABLE										
Minterms → / Prime Implicants ↓	0	1	2	4	6	8	9	11	13	15
B'C' (0,1,8,9)	1	1				1	1			
A'D'(0,2,4,6)	1		1		1					
AD(9,11,13,15)							1	1	1	1
Simplified Boolean function = B'C' + A'D' + AD										

- ✓ B'C' is in simplified function as minterm 1, 8 is only covered by B'C'. Similarly, minterms 2,4,6 are only covered by A'D' and minterms 11,13,15 are only covered by AD.

Logic Minimization

Example: Minimize the given function by Quine-McCluskey method
$F(A,B,C,D,E,F,G) = \sum m(20,28,52,60)$

Solution:

TABLE-1

Group	Minterms	A	B	C	D	E	F	G
0	20	0	0	1	0	1	0	0
1	28	0	0	1	1	1	0	0
	52	0	1	1	0	1	0	0
2	60	0	1	1	1	1	0	0

TABLE-2

Group	Pair	A	B	C	D	E	F	G
0	(20,28)	0	0	1	–	1	0	0
	(20,52)	0	–	1	0	1	0	0
1	(28,60)	0	–	1	1	1	0	0
	(52,60)	0	1	1	–	1	0	0

TABLE-3

Group	Quad	A	B	C	D	E	F	G
0	(20,28,52,60)	0	–	1	–	1	0	0

Prime Implicants Table

Minterms → / Prime Implicants ↓	20	28	52	60
A'CEF'G'(20,28,52,60)	1	1	1	1

Simplified Boolean Function = A'CEF'G'

- A'CEF'G' is in simplified function as it is the only prime implicant that covers all minterms.

Logic Minimization

Shannon's Reduction Using Relay Switches:

- It refers to Claude Shannon's pioneering work in applying Boolean algebra to simplify relay and switching circuits.

Key Concepts of Shannon's Reduction:

1. **Boolean Algebra Representation:** Shannon represented relay circuits using Boolean variables, where:
   - ✓ A closed switch (conducting) is represented by **1** (TRUE).
   - ✓ An open switch (non-conducting) is represented by **0** (FALSE).

2. **Logic Operations with Switches:**

   AND Operation (Series Connection): Two switches in series conduct if both are closed, corresponding to the logical AND operation.

   OR Operation (Parallel Connection): Two switches in parallel conduct if either is closed, corresponding to the logical OR operation.

   NOT Operation (Normally Closed Switch): A normally closed relay contact represents a logical NOT operation.

Logic Minimization

3. Circuit Simplification using Boolean Algebra:

- Shannon applied Boolean algebra rules (such as De Morgan's laws, distribution, and absorption) to minimize the number of relays and switches in a circuit.
- This reduced the complexity, cost, and power consumption of relay-based computing systems.

4. Foundation for Digital Logic Design:

- His work laid the foundation for digital circuit design, influencing the development of modern computers and digital systems.

Example of Shannon's Reduction:

Consider a relay circuit implementing the function:

$$F=AB+AC$$

Using Boolean simplification:

$$F=A(B+C)$$

This means that instead of using two separate series circuits for AB and AC, we can use a single switch A controlling a parallel combination of B and C, reducing the number of relays.

Session 6

Lecture: Logical Hazards

- ✓Hazards and glitches
- ✓Hazard elimination

Logical Hazards

Hazards and Glitches:

- **Logical hazards** are unwanted, brief glitches (momentary output changes) in a digital circuit due to different propagation delays in combinational logic.
- These hazards can cause errors in sequential circuits, especially in **asynchronous systems** where glitches might be interpreted as valid signals.

Types of Logical Hazards:

There are three main types of hazards:

1. **Static Hazards:** It occurs when an output should remain constant, but due to propagation delays, it momentarily changes (glitches).
    1. Two types:

        Static-1 Hazard: The output should stay 1, but it briefly becomes 0.

        Static-0 Hazard: The output should stay 0, but it briefly becomes 1.
    2. Cause: Different paths in the circuit having unequal delay times.
    3. Example: Suppose a function F=AB+A'C

    If A changes from 1 to 0, both paths might briefly be 0 before stabilizing.

Logical Hazards

Types of Logical Hazards:

2. Dynamic Hazards:

- It occurs when an output should change once, but instead it changes multiple times (oscillates briefly).
- **Cause**: Multiple paths with different delays, causing multiple transitions instead of one.

3. Essential Hazards:

- It occurs in asynchronous sequential circuits when signals arrive at different times, causing unintended state changes.
- **Cause**: Feedback loops where input changes are not synchronized.

Example of Static Hazard

Consider a function: F=AB+AC

Scenario:

Inputs: A = 1, B = 1, C = 1 → Output F = 1.

Now, A changes from 1 to 0.

Ideally, the output should remain stable at **0**, but due to different delays in different paths, F might **glitch** and momentarily become **1**.

Logical Hazards

Hazard Elimination:

1. Use Redundant Logic Terms (Hazard Covering)

- Modify the function to include extra terms ensuring no glitches.
- Example: F=AB+AC+BC (adding BC removes the hazard).

2. Use Synchronous Circuits

- Clocked systems avoid hazards by only sampling signals at stable points.

3. Minimize Delay Variations

- Use equal-length paths or buffering to match delays.

4. Use Karnaugh Map (K-map) to Identify Hazards

- Look for adjacent groups of 1s that are not fully covered.

❖ Logical hazards are crucial in **high-speed circuits** and **asynchronous systems**. By **adding redundant logic, synchronizing circuits, and matching delays**, hazards can be mitigated, ensuring stable digital circuit performance.

Session 7

Lecture: Logic Families

- ✓ Introduction to different Logic families with their sub-families/types.
- ✓ TTL – NAND using TTL with working, different types of TTL families with features, usage.
- ✓ ECL – NAND using ECL with working, different types of ECL families with features, usage.
- ✓ CMOS – NAND using CMOS with working, different types of CMOS families with features, usage.
- ✓ Comparison of speed and power of different families and sub families/types.

Logic Families

- A **logic family** is a group of digital circuits built using similar electronic components and having compatible electrical characteristics.
- Different logic families have varying speed, power consumption, noise immunity, and voltage levels.

Types of Logic Families:

Logic families are broadly classified into **Bipolar** and **MOS** families.

1. Bipolar Logic Families (BJTs - Bipolar Junction Transistors)

These families use **bipolar transistors** for switching.

Family	Characteristics
Resistor-Transistor Logic (RTL)	Earliest logic family, high power consumption, slow speed.
Diode-Transistor Logic (DTL)	Improved over RTL, but still slow.
Transistor-Transistor Logic (TTL)	Most common bipolar logic family, moderate speed and power.
Emitter-Coupled Logic (ECL)	Very fast, high power consumption, used in high-speed computing.

Logic Families

2. MOS Logic Families (FETs - Field Effect Transistors)

These families use **MOSFETs** instead of BJTs, offering lower power consumption.

Family	Characteristics
PMOS (P-channel MOS)	Early MOS technology, slow, high power.
NMOS (N-channel MOS)	Faster than PMOS but still power-hungry.
CMOS (Complementary MOS)	Low power, high noise immunity, widely used in VLSI.

Comparison of Major Logic Families:

Feature	TTL	CMOS	ECL
Power Consumption	High	Low	Very High
Speed	Moderate	High	Very High
Noise Immunity	Moderate	High	Low
Voltage Levels	5V	3V–15V	-5.2V

Logic Families

Transistor-Transistor Logic (TTL):

- TTL is a **bipolar junction transistor (BJT)-based** logic family widely used in digital circuits before CMOS became dominant.

Characteristics of TTL:

✓ Uses **BJTs** for switching
✓ Operates at **5V logic levels**
✓ Faster than early logic families (like DTL, RTL)
✓ Moderate power consumption compared to ECL

TTL Logic Levels (Standard 5V):

Logic State	Voltage Level
Logic 0 (LOW)	0V – 0.8V
Logic 1 (HIGH)	2V – 5V

Logic Families

TTL – NAND Using TTL With Working:

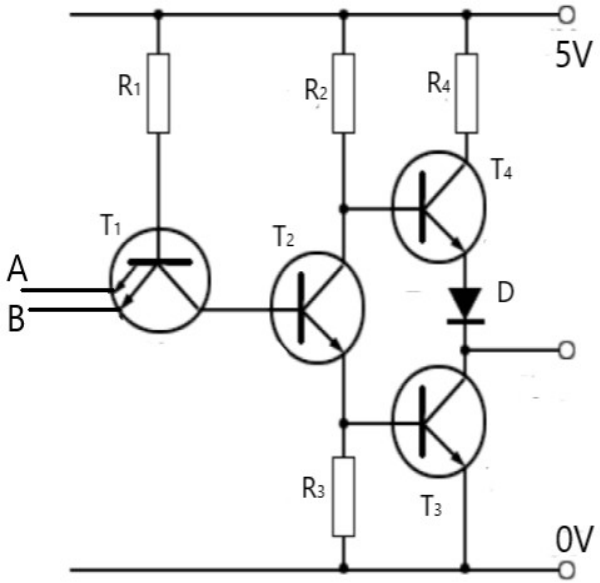
- The **NAND gate** is one of the most fundamental logic gates, and in **Transistor-Transistor Logic (TTL)**, it is implemented using **BJTs (Bipolar Junction Transistors)**.

Circuit Diagram of TTL NAND Gate:

- A **2-input TTL NAND gate** can be implemented using **multiple BJTs** arranged in a totem-pole configuration.

Basic Circuit Components:

- **Multiple NPN transistors** for switching.
- **Diodes and resistors** for voltage control and biasing.
- **Totem-pole output stage** for improved switching speed.



Logic Families

TTL – NAND Using TTL With Working:

The TTL NAND gate has **three stages**:

(i) Input Stage (Multiple-Emitter Transistor)

- A **multi-emitter transistor (T1)** is used.
- The emitters act as inputs (**A and B**).
- This configuration allows the gate to handle multiple inputs efficiently.

(ii) Phase-Splitter Stage

- A second transistor (**T2**) acts as a **phase splitter** to drive the output stage.
- This ensures proper signal inversion.

(iii) Totem-Pole Output Stage

- Two transistors (**T3 and T4**) form the **totem-pole** structure.
- **T4** helps pull the output HIGH, while **T3** pulls it LOW.
- This improves speed by reducing transition time.

Logic Families

TTL – NAND Using TTL With Working:

Explanation:

- If **any input is 0**, at least one transistor remains OFF, and the output stays HIGH (1).
- If **both inputs are 1**, all transistors conduct, and the output becomes LOW (0).

Case 1: A = 0, B = 0 → Output Y = 1

- Both inputs are LOW (0).
- Transistor **T1** is OFF, preventing Q2 from turning ON.
- **T4 turns ON**, pulling the output HIGH.

Case 2: A = 0, B = 1 or A = 1, B = 0 → Output Y = 1

- One input is HIGH, one is LOW.
- The multi-emitter transistor does not fully conduct.
- **T4 remains ON**, keeping the output HIGH.

Case 3: A = 1, B = 1 → Output Y = 0

- Both inputs are HIGH (1).
- **T1 fully conducts**, activating T2 and T3.
- **T3 turns ON**, pulling the output LOW.

Input A	Input B	Output Y (A NAND B)
0	0	1
0	1	1
1	0	1
1	1	0

Logic Families

TTL Subfamilies:

✓ TTL has several variations to optimize **speed, power consumption, and noise immunity**.

TTL Family	Speed (Propagation Delay)	Power Consumption	Primary Usage
Standard TTL (74)	~10ns	~10mW	Basic logic circuits
Low-Power TTL (74L)	~33ns	~1mW	Battery-powered devices
High-Speed TTL (74H)	~6ns	~22mW	High-speed data processing
Schottky TTL (74S)	~3ns	~20mW	Fast processors, memory systems
Low-Power Schottky TTL (74LS)	~9ns	~10mW	Microcontrollers, automation
Advanced Schottky TTL (74AS)	~1.5ns	~30mW	High-performance computing
Advanced Low-Power Schottky TTL (74ALS)	~4ns	~1mW	Low-power logic circuits
Fast TTL (74F)	~1.5ns	~5mW	High-speed networking & telecom

Logic Families

TTL Subfamilies:

- **74, 74L, 74H** → Used for **basic logic and control applications**.
- **74S, 74LS** → **Faster processing, industrial automation**.
- **74AS, 74ALS, 74F** → **High-speed computing & networking**.

Advantages of TTL

✓ Faster than early logic families like RTL and DTL.
✓ Robust operation in noisy environments.
✓ Large variety of ICs available.

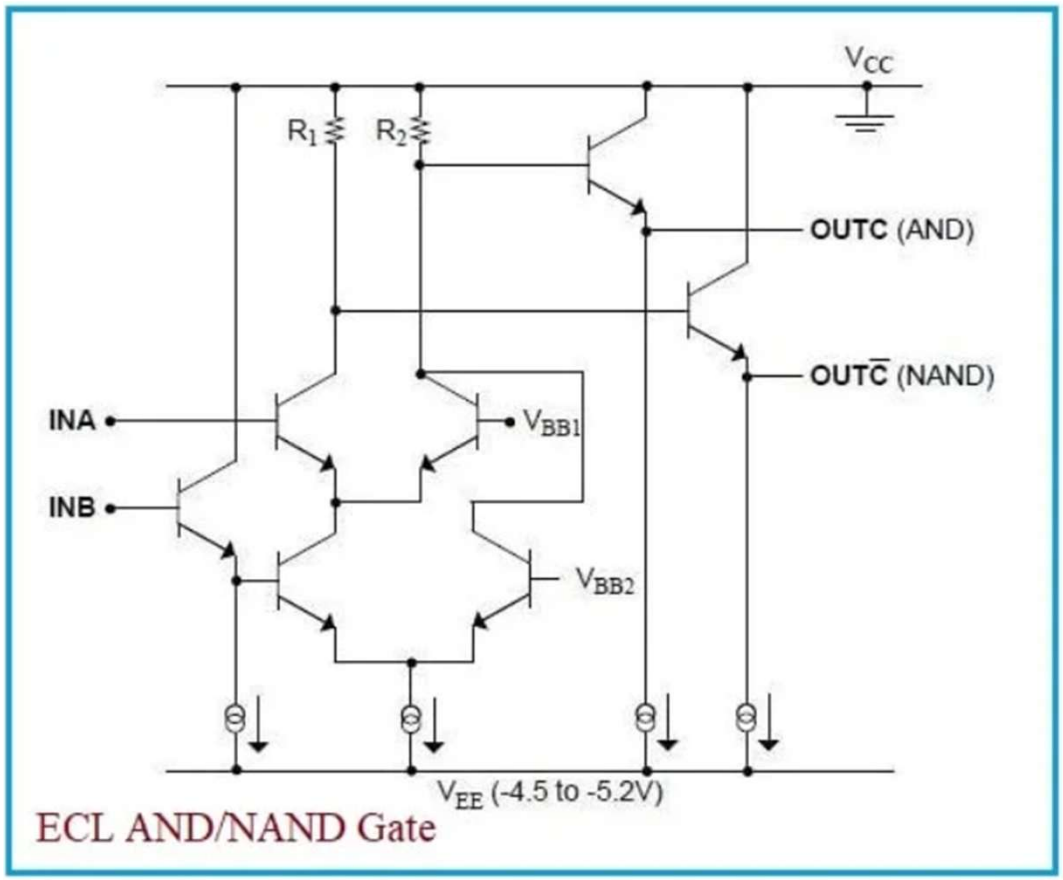
Disadvantages of TTL

✗ Higher power consumption compared to CMOS.
✗ Limited scaling for VLSI applications.
✗ Lower input impedance than CMOS (draws more current).

Logic Families

ECL – NAND Using ECL With Working:

- **Emitter-Coupled Logic (ECL)** is a high-speed digital logic family that operates by using differential amplifiers instead of saturating transistors (as in TTL). ECL is known for its **high-speed operation** and **low voltage swing**, making it ideal for applications requiring very fast switching times.



ECL AND/NAND Gate

Logic Families

ECL – NAND Using ECL With Working:

- An ECL NAND gate is constructed using multiple transistors to achieve the logic functionality. The basic structure involves differential pairs and additional transistors for signal inversion and combination.

- Here, two differential pairs are used, each receiving one of the inputs (A and B). Additional transistors are connected to the collectors of the differential pairs to form a logical AND function. The outputs of these transistor stacks are then combined to produce the NAND gate's final output.

- The final output is typically taken from an emitter-follower stage, which provides a low-impedance output for driving subsequent stages.

Advantages of ECL

✓ **Very high-speed operation** (low propagation delay, ~1ns).
✓ **No transistor saturation**, reducing switching time.
✓ **Low output voltage swing** minimizes power dissipation.

Disadvantages of ECL

✕ **High power consumption** (continuous current flow).
✕ Requires a **negative power supply (-5.2V in standard ECL)**.
✕ More complex than TTL and CMOS circuits.

Logic Families

Different Types of ECL Families With Features & Usage:

- **Emitter-Coupled Logic (ECL)** is known for its **high-speed operation**, making it suitable for **high-frequency applications**. Over time, different **ECL families** have been developed to improve performance, reduce power consumption, and enhance compatibility.

Different Types of ECL Families:

- Standard ECL (10K ECL)
- 100K ECL (Advanced ECL)
- Low-Power ECL (LP-ECL)
- Positive ECL (PECL)
- Low-Voltage Positive ECL (LVPECL)

Logic Families

Different Types of ECL Families With Features & Usage:

ECL Family	Voltage Supply	Propagation Delay	Power Consumption	Primary Use
10K ECL	-5.2V	~2ns	High	Supercomputers, early digital logic
100K ECL	-5.2V	~1ns	Moderate	Military, high-speed computing
LP-ECL	-5.2V	~1.5ns	Low	Portable high-speed electronics
PECL	+5V / +3.3V	~1ns	Moderate	Clock distribution, networking
LVPECL	+3.3V / +2.5V	~0.5ns – 1ns	Low	Fiber-optics, wireless networks

Logic Families

Complementary Metal-Oxide Semiconductor (CMOS):

- CMOS logic uses **MOSFETs** instead of BJTs, making it more power-efficient.

Characteristics of CMOS:

✓ **Very low power consumption** (only draws power during switching).
✓ **High noise immunity** compared to TTL.
✓ **Supports a wide voltage range (3V – 15V)**.
✓ **Easier to scale for VLSI integration**.

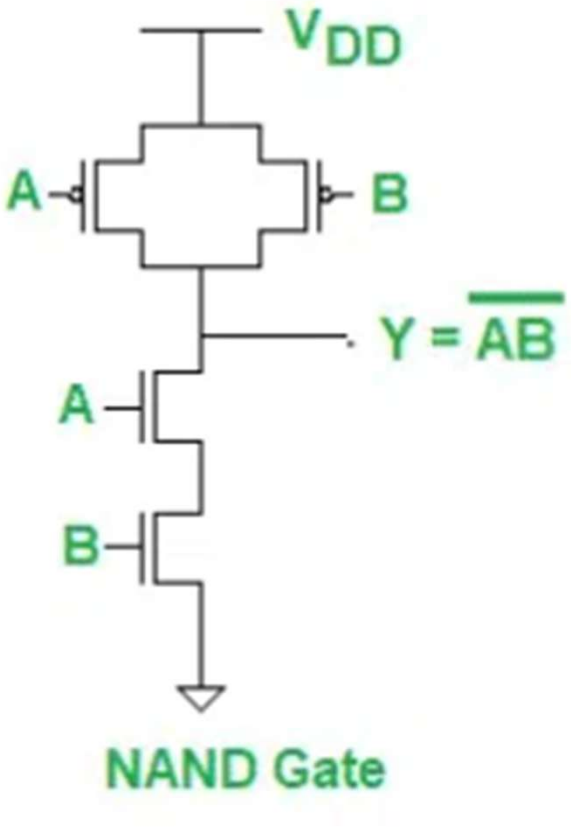
CMOS Logic Levels (for 5V operation):

Logic State	Voltage Level
Logic 0 (LOW)	0V – 1.5V
Logic 1 (HIGH)	3.5V – 5V

Logic Families

CMOS – NAND Using CMOS With Working:

- A **CMOS NAND gate** is built using **Complementary Metal-Oxide-Semiconductor (CMOS) technology**, which utilizes both **PMOS** and **NMOS transistors** to achieve low power consumption and high speed.
- In CMOS NAND, NMOSs are in series and PMOSs are in parallel and opposite in CMOS NOR.



Input A	Input B	Output Y (A NAND B)
0	0	1
0	1	1
1	0	1
1	1	0

Logic Families

CMOS Subfamilies:

✓ CMOS families have evolved to improve performance and compatibility with TTL.

CMOS Family	Voltage Range	Speed (Propagation Delay)	Power Consumption	Primary Usage
4000 Series (Standard CMOS)	3V – 18V	~100ns	Very Low	Simple logic circuits, space applications
74HC / 74HCT (High-Speed CMOS)	2V – 6V / 4.5V – 5.5V	~10ns	Low	Consumer electronics, TTL interfacing
74AC / 74ACT (Advanced CMOS)	2V – 6V / 4.5V – 5.5V	~5ns	Moderate	High-speed computing, networking
LVC / LV / AUC (Low-Voltage CMOS)	0.8V – 3.3V	<2ns	Very Low	Mobile devices, IoT, wearable tech
74LV / 74LVC (Low-Power CMOS)	1.8V – 3.3V	~3-5ns	Very Low	Battery-powered devices, medical electronics
74ABT / 74BCT (BiCMOS)	5V	~4ns	High	High-performance computing, RF circuits

Logic Families

CMOS Subfamilies:

- **4000 Series** → For **basic low-power logic circuits**.
- **74HC / HCT** → For **general high-speed applications**.
- **74AC / ACT** → For **faster processing and industrial use**.
- **LVC / LV / AUC** → For **mobile, battery-powered, and IoT devices**.
- **BiCMOS** → For **high-performance computing and analog-mixed circuits**.

Advantages of CMOS

✓ **Extremely low power consumption** (ideal for battery-powered devices).
✓ **High input impedance** (draws less current).
✓ **Higher integration density** (more transistors per chip).
✓ **Operates at lower voltages**, making it suitable for modern processors.

Disadvantages of CMOS

✕ Slower than TTL in earlier versions (modern versions have improved speed).
✕ More susceptible to **ESD (Electrostatic Discharge)**.
✕ Higher **propagation delay** in older CMOS families.

Logic Families

Where are TTL and CMOS Used?

- **TTL** is still used in applications requiring **high-speed switching** and **rugged operation** (e.g., industrial automation).
- **CMOS** is used in **modern VLSI, microprocessors, and portable electronics** due to its **low power consumption**.

TTL vs. CMOS: A Comparison

Feature	TTL (74 series)	CMOS (4000, 74HC, 74HCT)
Technology	Uses BJTs	Uses MOSFETs
Power Consumption	High (mW range per gate)	Very low (µW range per gate)
Speed	Fast (10ns range)	Slower in early versions, now very fast
Voltage Range	Fixed at 5V	Wide range (3V–15V)
Noise Immunity	Moderate	High
ESD Sensitivity	Low	High (requires handling precautions)
Scaling for VLSI	Limited	Excellent

Session 8

Lecture: Tristate and Other Circuits

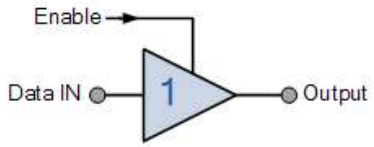
- ✓Tristate
- ✓Muller-C
- ✓AOI (And-Or-Inverter)
- ✓Design of Logic Gates with CMOS technology: NOT, NAND, NOR, XOR.

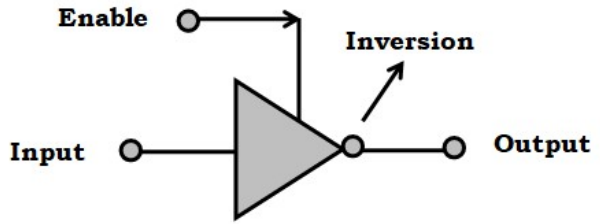
Tristate and Other Circuits

Tristate Circuits:

- ✓ A **Tri-State Circuit** (also called **Three-State Logic**) is a type of digital circuit where the output can exist in **three different states**:
  - ❖ **Logic HIGH (1)**
  - ❖ **Logic LOW (0)**
  - ❖ **High Impedance (Z)** → Acts as if the output is disconnected (floating).
- ✓ This third state (**High Impedance or Z-state**) allows multiple devices to share the same output line without interference, which is useful in buses and communication systems.

Active "HIGH" Tri-state Buffer

Symbol	Truth Table		
 Tri-state Buffer	Enable	IN	OUT
	0	0	Hi-Z
	0	1	Hi-Z
	1	0	0
	1	1	1
Read as Output = Input if Enable is equal to "1"			



Enable	Input	Output
0	0	Hi-Z
0	1	Hi-Z
1	0	1
1	1	0

Tristate and Other Circuits

Applications of Tri-State Circuits:

✅ Bus Systems

- In microprocessors, **data buses** need multiple devices to communicate.
- Tri-state buffers allow **one device to transmit data at a time**, while others stay in High Impedance mode.

✅ Multiplexing

- Used in **memory chips and registers** to connect multiple devices to a shared line.

✅ Microprocessors & Microcontrollers

- Control signals use tri-state logic for **bidirectional data flow**.

✅ I/O Ports

- Used in **bidirectional communication** like USB, SPI, and I2C protocols.

Tristate and Other Circuits

Muller-C:

❖ The **Muller-C Element** (also called **C-Gate**) is a fundamental asynchronous logic circuit used in **self-timed digital systems**.

Function of Muller-C Element:

- The **Muller-C Element** is a **state-holding logic gate** that outputs a value only when all inputs agree. If the inputs differ, it retains its previous state.

Input A	Input B	Previous Output (Y_prev)	New Output (Y)
0	0	X	0
0	1	Y_prev	Y_prev
1	0	Y_prev	Y_prev
1	1	X	1

- If A = B, the output follows A (or B).
- If A ≠ B, the output retains its previous value (Y_prev).

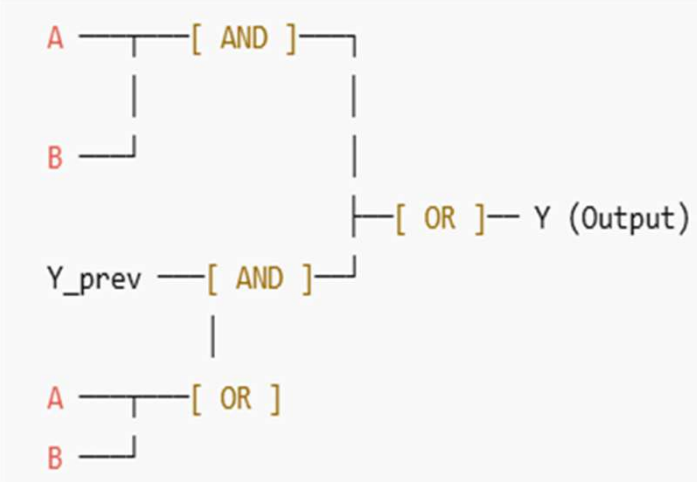
Tristate and Other Circuits

Muller-C:

- A Muller-C Element can be constructed using **AND, OR, and feedback connections.**

- $Y=(A\cdot B)+(Yprev\cdot(A+B))$

- If **A and B are both 1**, output becomes 1.
- If **A and B are both 0**, output becomes 0.
- If **A ≠ B**, the output **remains unchanged** (Y_prev).



- **Two AND gates** detect when both inputs are 1 or when the previous output should be maintained.
- **An OR gate** combines these conditions to produce the output.

Tristate and Other Circuits

Applications of Muller-C Element:

✅ **Asynchronous Circuits:** Used in **clockless logic** where operations rely on signal completion rather than a clock pulse.

✅ **Pipeline Control:** Controls data flow in **self-timed pipelines**, ensuring safe transfer between stages.

✅ **Handshake Protocols:** Used in **data communication** where acknowledgments must be correctly sequenced.

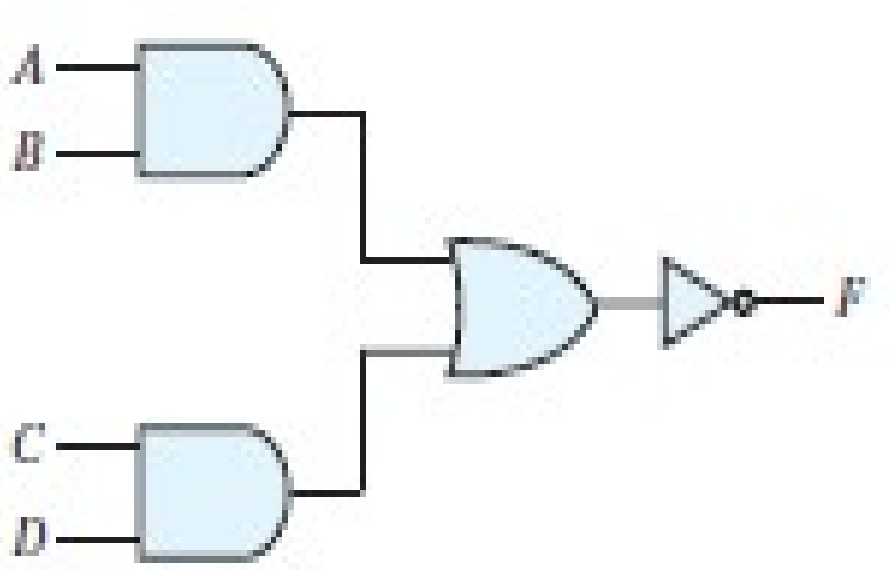
✅ **Low-Power Digital Design:** Since it eliminates the need for a clock, it helps reduce **power consumption**.

Tristate and Other Circuits

AOI Circuits:

- ✓ The **AOI logic** is a fundamental digital circuit configuration that combines **AND, OR, and Inverter (NOT) gates** into a single compact structure.
- ✓ It is widely used in **VLSI design** to optimize circuit performance by reducing transistor count and power consumption.
- The logic function

  F = (AB + CD)' = (A' + B')(C' + D') is called an AND–OR–INVERT function.

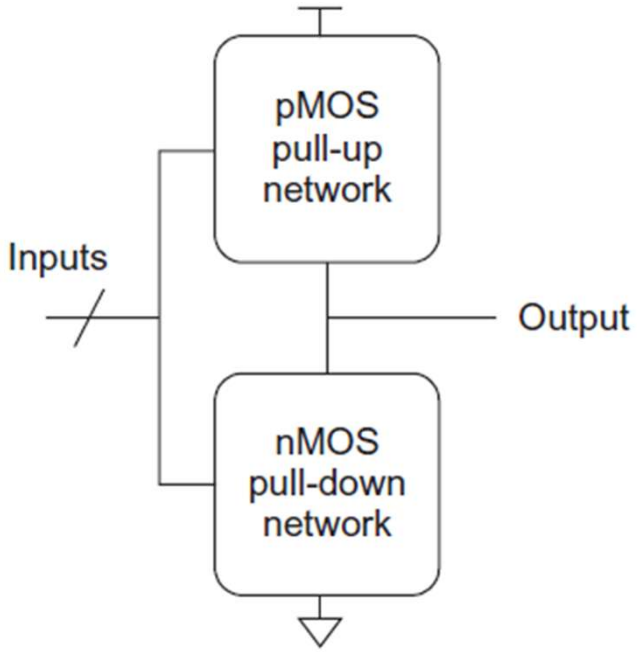


CMOS Logic Circuits

- CMOS logic circuits utilize both NMOS and PMOS transistors to implement various digital logic functions.

CMOS Logic Gates:

- ✓ The Inverter, NAND, NOR gates are examples of static CMOS logic gates, also called complementary CMOS gates.
- ✓ In general, a static CMOS gate has an nMOS pull-down network to connect the output to 0 (GND) and pMOS pull-up network to connect the output to 1 (VDD) as shown in below Figure.
- ✓ The networks are arranged such that one is ON and the other OFF for any input pattern.



CMOS Logic Circuits

CMOS Logic Gates:

✓ The pull-up and pull-down networks in the inverter each consist of a single transistor.

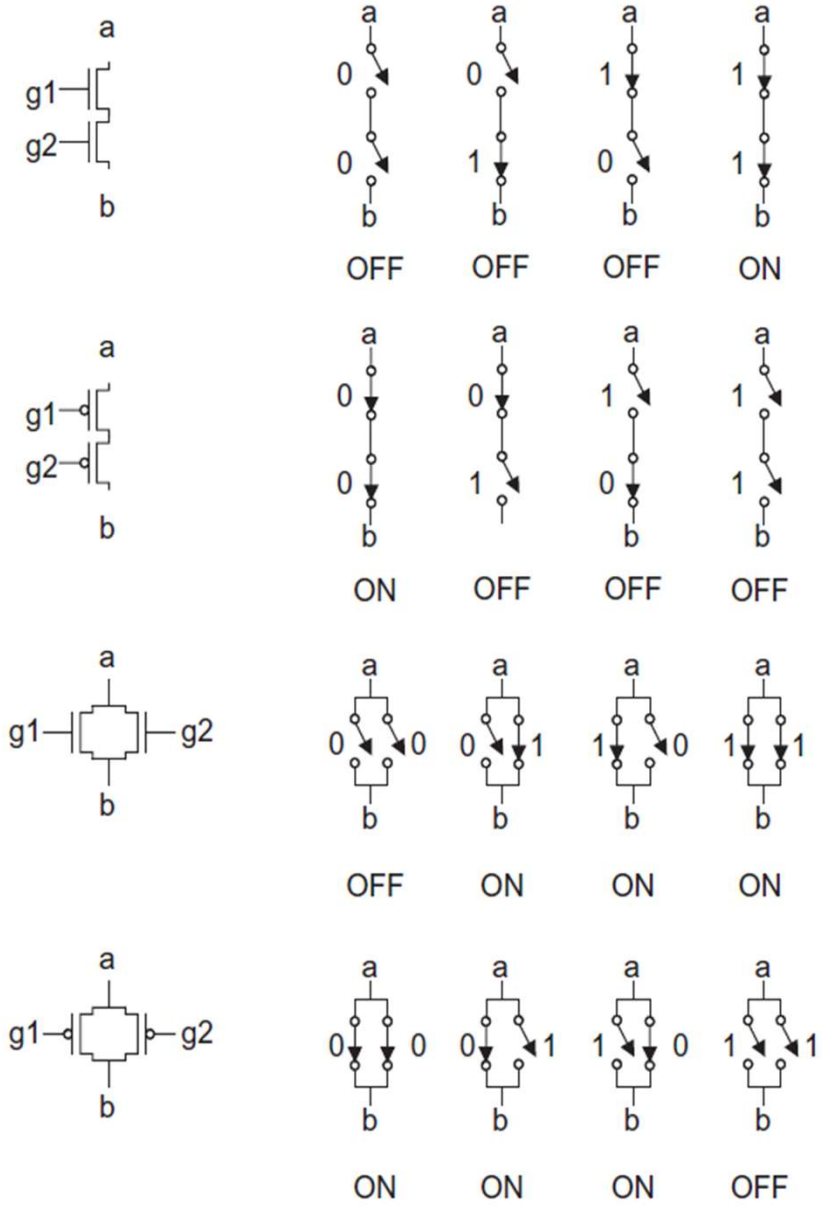
✓ The NAND gate uses a series pull-down network and a parallel pullup network and reverse is in NOR gate.

✓When both pull-up and pull-down are OFF, the high impedance or floating Z output state results. This is of importance in multiplexers, memory elements, and tristate bus drivers.

✓The crowbarred (or contention) X level exists when both pull-up and pull-down are simultaneously turned ON.

✓Contention between the two networks results in an indeterminate output level and dissipates static power. It is usually an unwanted condition.

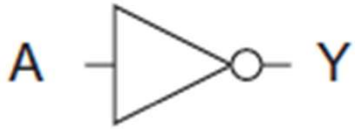
	pull-up OFF	pull-up ON
pull-down OFF	Z	1
pull-down ON	0	crowbarred (X)

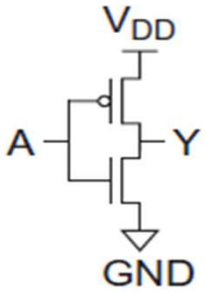


CMOS Logic Circuits

Basic CMOS Logic Gates: NOT (Inverter), NAND, NOR etc

NOT Gate:

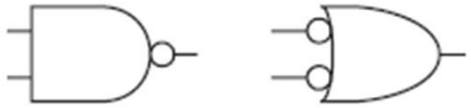


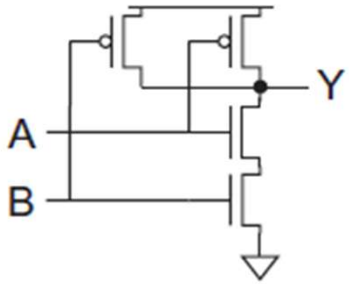


A	Y
0	1
1	0

NAND Gate:

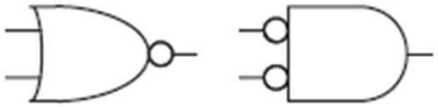
2-input CMOS NAND gate consists of two series nMOS transistors between Y and GND and two parallel pMOS transistors between Y and VDD.

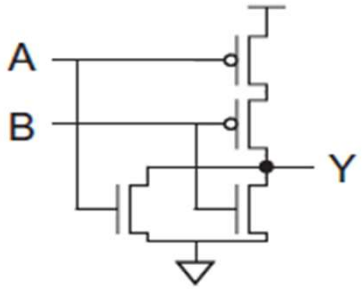




A	B	Pull-Down Network	Pull-Up Network	Y
0	0	OFF	ON	1
0	1	OFF	ON	1
1	0	OFF	ON	1
1	1	ON	OFF	0

NOR Gate: Connection of series pMOS and parallel nMOS.





A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

CMOS Logic Circuits

Design NOT Gate with CMOS Technology:

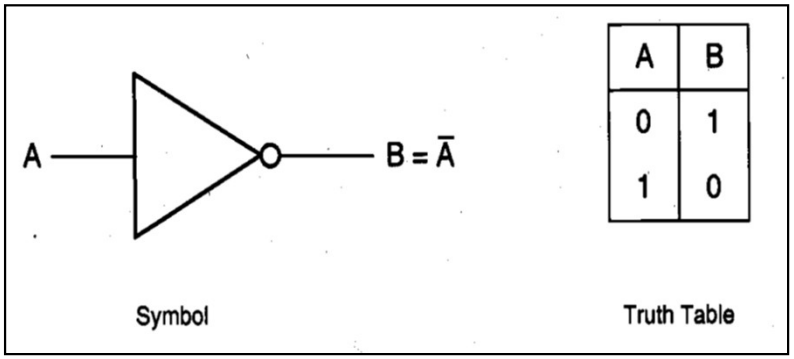
✓ A **CMOS inverter** consists of:

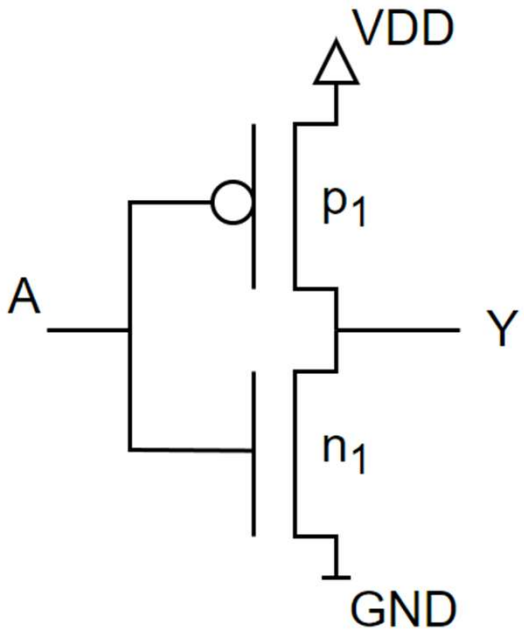
✅ 1 PMOS transistor (pull-up network - PUN)

✅ 1 NMOS transistor (pull-down network - PDN)

✓ The **PMOS and NMOS transistors are connected in a complementary manner**:

- The **PMOS transistor is connected to VDD** (logic HIGH).
- The **NMOS transistor is connected to GND** (logic LOW).



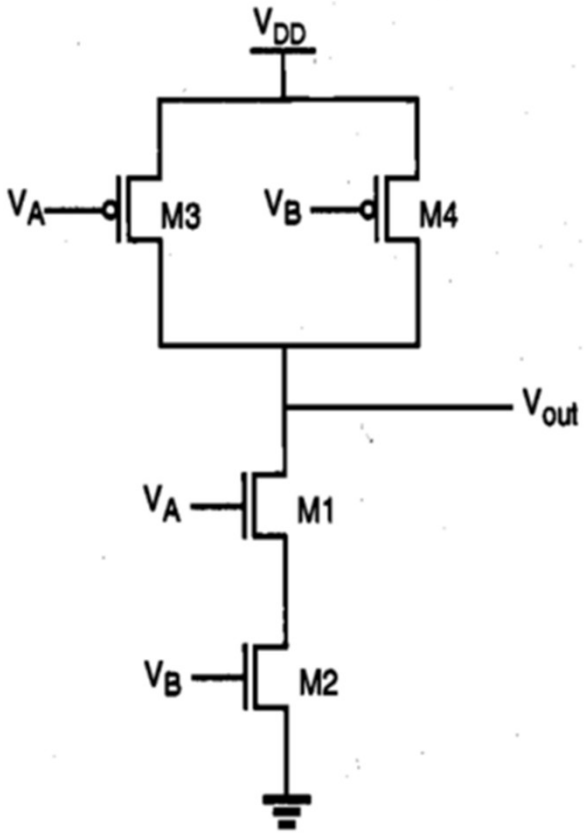


CMOS Logic Circuits

Design NAND Gate with CMOS Technology:

- A **CMOS NAND gate** is implemented using:
  - ✅ Two PMOS transistors (in parallel) in the pull-up network (PUN)
  - ✅ Two NMOS transistors (in series) in the pull-down network (PDN)

Input A	Input B	Output Y (A NAND B)
0	0	1
0	1	1
1	0	1
1	1	0

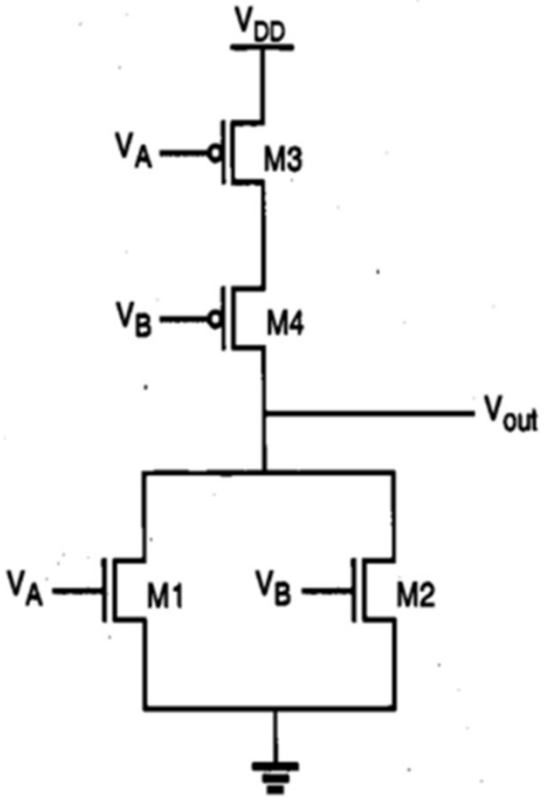


CMOS Logic Circuits

Design NOR Gate with CMOS Technology:

- A **CMOS NOR gate** consists of:
  - ✅ **Two PMOS transistors in series in the pull-up network (PUN)**
  - ✅ **Two NMOS transistors in parallel in the pull-down network (PDN)**

Input A	Input B	Output Y (A NOR B)
0	0	1
0	1	0
1	0	0
1	1	0



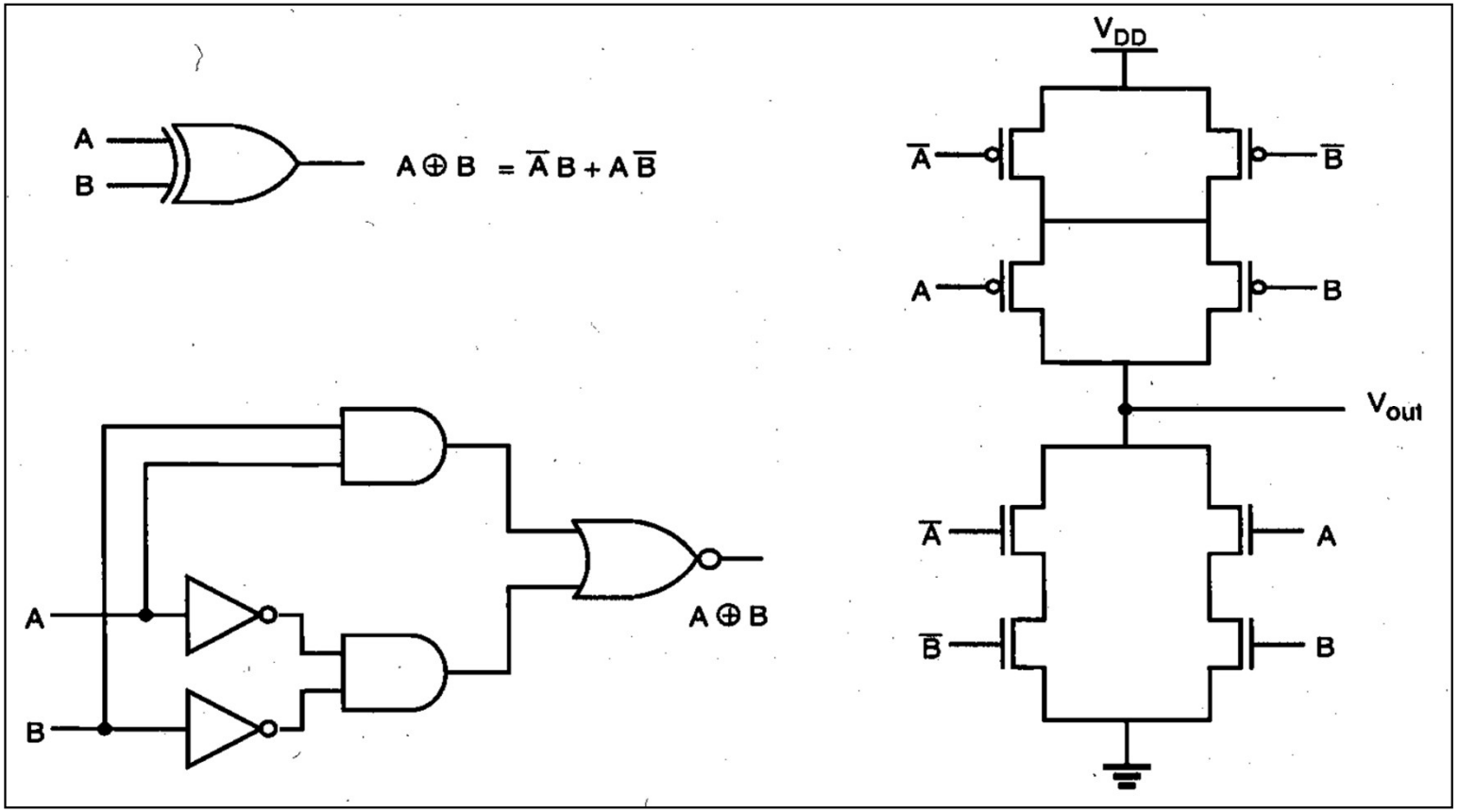
CMOS Logic Circuits

Design XOR Gate with CMOS Technology:

✅ **A combination of AND, OR, and NOT logic using CMOS transistors**

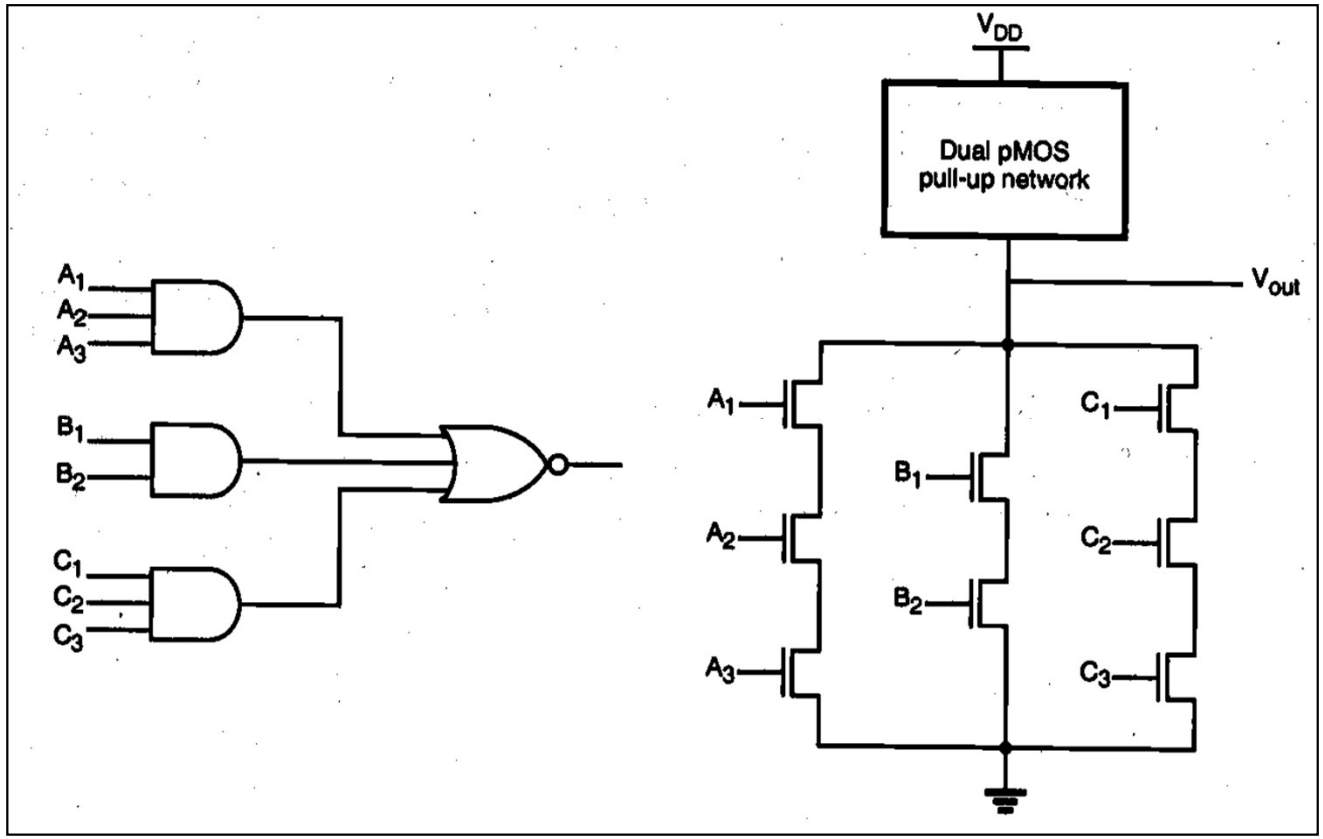
The **CMOS implementation** uses **two main logic parts**:

- **Pull-Up Network (PUN) – PMOS transistors** to drive the output HIGH.
- **Pull-Down Network (PDN) – NMOS transistors** to drive the output LOW.



CMOS Logic Circuits

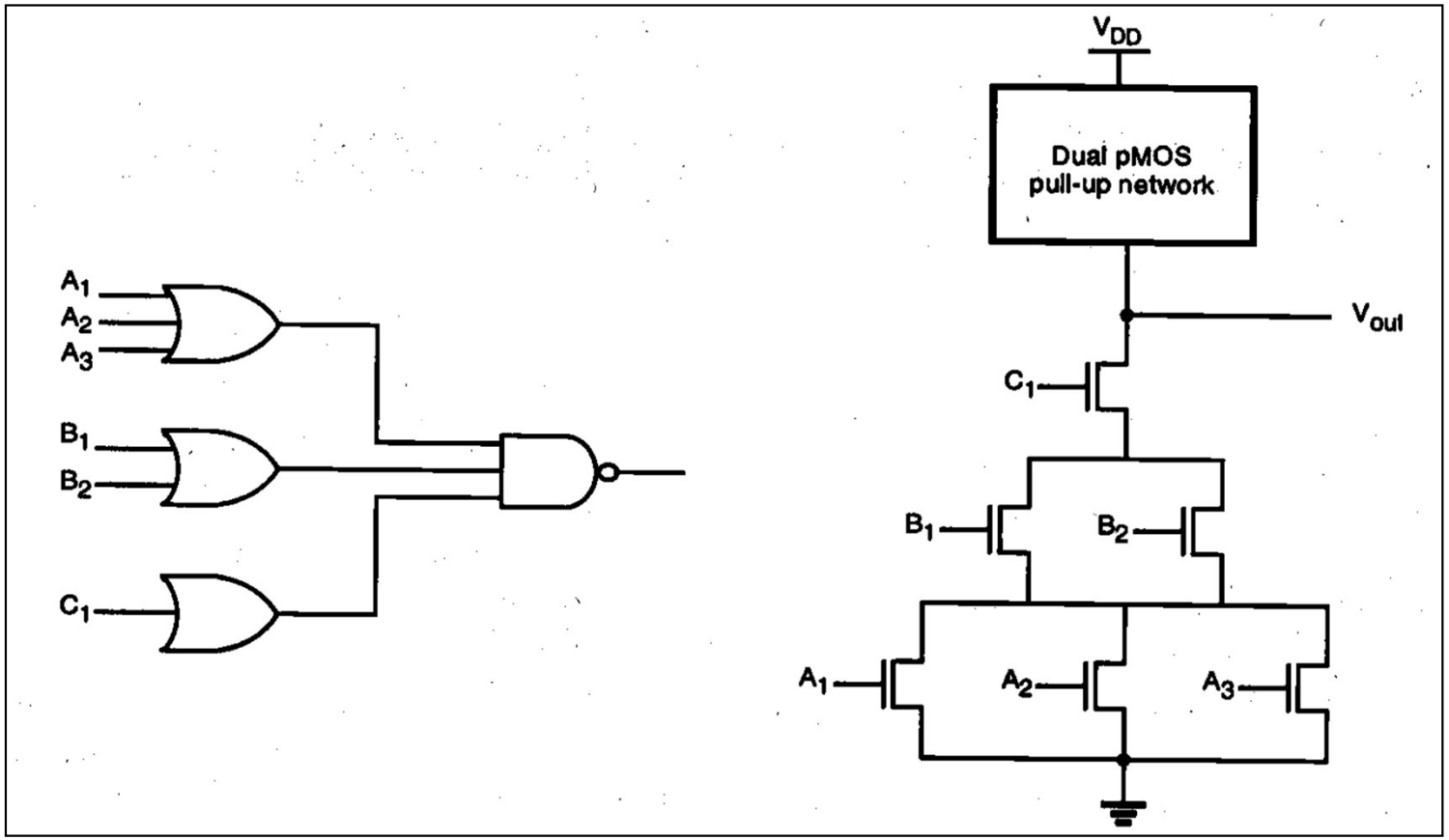
- The pull-down net of the AOI gate consists of parallel branches of series-connected nMOS driver transistors. The corresponding p-type pull-up network can simply be found using the dual-graph concept.



An AND-OR-INVERT (AOI) gate and the corresponding pull-down net

CMOS Logic Circuits

- The pull-down net of the OAI gate consists of series branches of parallel-connected nMOS driver transistors, while the corresponding p-type pull-up network can be found using the dual-graph concept.



An OR-AND-INVERT (OAI) gate and the corresponding pull-down net

THANK YOU